RISC-V Matrix Specification

Version v0.0.0, 2025-07-01: Draft

Table of Contents

Preamble

This document is in the Development state

Assume everything can change. This draft specification will change before being accepted as standard, so implementations made to this draft specification will likely not conform to the future standard.

Copyright and license information

Contributors

This RISC-V specification proposal has been contributed to directly or indirectly by:

ZhaoJiang, LeiKang, RuiqiXie, XuchaoGao, WenganShao, ZhiweiLiu, XianmiaoQu, JingQiu, MingxinZhao, YunhaiShang, XiaoyanXiang, ChenChen

Others are welcome to help improve the specification.

We will be very grateful to the huge number of other people who will have helped to improve this specification through their comments, reviews, feedback and questions.

Change List

The change list of v0.5.0 is as follows:

Number	Content
1	csr address change
2	add new csr, xmxadesc/xmxbdesc
3	misa change for mhp/mmbf20f32/mmf4f16/mmf4bf16/mmf4f32/mmmxf8/mmmxf4/mmmxf8mxf4
4	xmfp8sat change to xmsaten, add integer saturation mode for mmacc instructions
5	add transposed load/store and prefetch load instruction
6	add int4 to fp8 conversion
7	add mmbf20f32
8	add mmf4f32/mmf4f16/mmf4bf16, including fp4 and fp8 conversion
9	add mmbf20f32
10	add microscaling format support, mmmxf8/mmmxf4/mmmxf8mxf4
11	mnemonic change (1)fmmacc to mfmacc
12	mnemonic change (2)mmaqa to mmacc.w.b
13	mnemonic change (3)pmmaqa to mmacc.w.p
14	mnemonic change (4)mld/st<b/h/w/d> to mld/st<e8/e16/e32/e64>
15	mnemonic change (5)e4/e5 to e4m3/e5m2
16	define the cNaN to 0x7f for e4m3 and e5m2
17	add mfmacc.s.mxe2m1.ue4m3
18	change mx blksize from 32/64/128/256 to 16/32/64/128

Chapter 1. Introduction

This document is a matrix extension proposal for RISC-V.

The extension chooses a decoupled architecture from the vector extension for the flexibility and implementation considerations. This extension defines extra matrix registers, including matrix multiplication instructions(C += A x BT), matrix load/store instructions, matrix element-wise operation instructions, and matrix misc instructions, which supports matrix multiplication and other basic matrix operation.

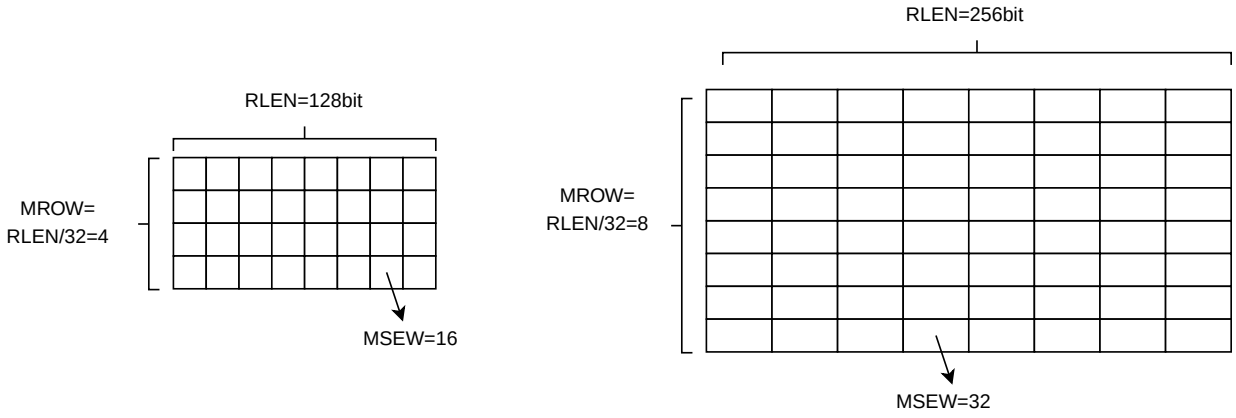
Chapter 2. Implementation-defined Constant Parameters

Each hart supporting a matrix extension defines one parameter:

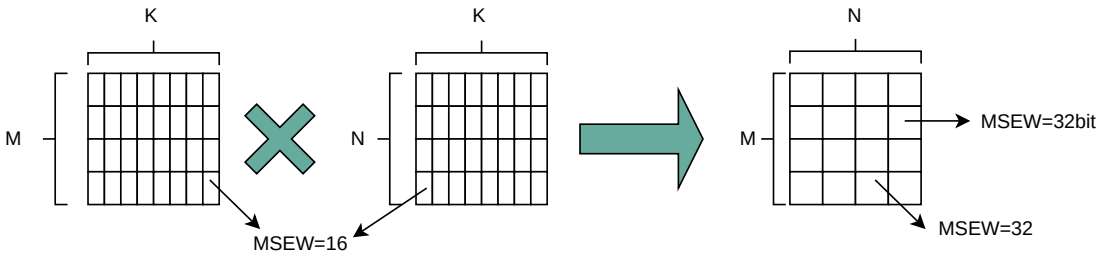
1. RLEN: represents the length of each register row in bits. It is a constant value in any implementation, and the available RLEN options are 128, 256, or 512 or more.

The following parameters can be deduced from RLEN:

1. MROW: matrix register row number, equals to RLEN/32, which can be 4, 8, or 16 or more.
2. MLEN: matrix register bits, equals to RLEN*MROW.
3. MSEW: the element size in bits in matrix register.



Matrix Register Shape



C = A * BT Matrix Multplication

The picture show examples for the matrix register shape and the `C += A x BT` matrix multiplication operation.

For `C += A x BT`, A matrix is stored in matrix register ms1 in normal format(M*K), and B matrix are stored in matrix register ms2 in transposed format(N*K). The shape of C matrix is M*N, which stored in md. M represents the row number of A matrix and C matrix, K represents the column number of A matrix and B matrix, and N represents row number of B matrix and column number of C matrix.

Chapter 3. Programmer’s Model

Matrix extension adds eight two-dimensional matrix registers and thirteen CSRs.

Address	Privilege	Name	Description
0x801	URW	xmrstart	matrix start row position
0x802	URW	xmcsr	matrix control and status register
0x803	URW	xmsize	matrix size configuration
0x810	URW	xmxadesc	matrix A microscaling format descriptor, if Theadmmx* supported
0x811	URW	xmxbdesc	matrix B microscaling format descriptor, if Theadmmx* supported
0xCC0	URO	xmlenb	matrix register size in byte, MLEN/8
0xCC1	URO	xrlenb	matrix register row size in byte, RLEN/8
0xCC2	URO	xmisa	matrix instruction subset

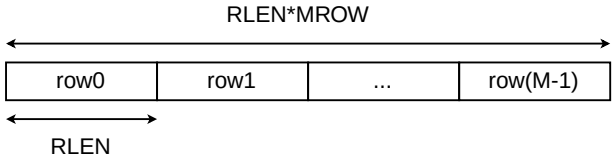
Matrix extention gives seperate CSR address to allow independent access of xmxrm/xmsat/xmfflags/xmfrm/xmsat which is also reflected as a field in xmcsr.

Address	Privilege	Name	Description
0x812	URW	xmxrm	fixed-point rounding mode
0x813	URW	xmxsat	fixed-point accrued saturation flag
0x814	URW	xmfflags	float-point accrued exception flags
0x815	URW	xmfrm	float-point rounding mode
0x816	URW	xmsaten	saturation mode enable for fp8 conversion or integer matrix multiplication instructions

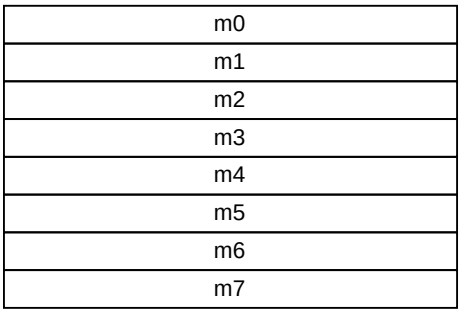
3.1. Matrix Registers

The matrix extension adds eight two-dimensional matrix registers named M0, M1, M2, M3, M4, M5, M6 and M7.

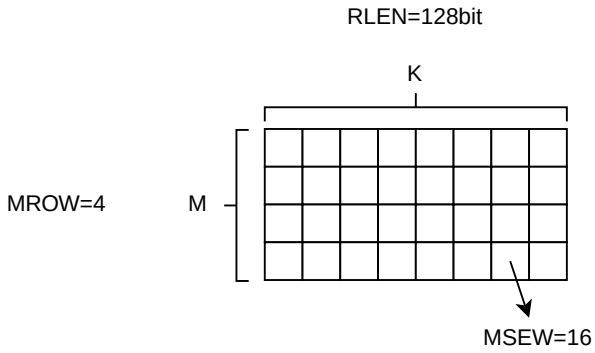
RLEN represents the length of each register row in bits. It is a constant value in an implementation, and the available RLEN options are 128, 256, or 512 or more. The matrix register row number is RLEN divided by 32, resulting in 4, 8, or 16 or more. Consequently, each matrix register comprises [M x K] MSEW elements, where M is calculated as RLEN divided by 32, K is calculated as RLEN divided by MSEW.



Matrix Register Structure



Matrix Registers



Matrix Register Shape

The size of the matrix registers varies as following.

RLEN(bit)	MSEW(bit)	M	K	MLEN(bit)
128	4	4	32	512
128	8	4	16	512
128	16	4	8	512
128	32	4	4	512
128	64	4	2	512
256	4	8	64	2048
256	8	8	32	2048
256	16	8	16	2048
256	32	8	8	2048
256	64	8	4	2048
512	4	16	128	8192
512	8	16	64	8192
512	16	16	32	8192
512	32	16	16	8192
512	64	16	8	8192

3.2. Matrix ISA Register, misa

The read-only XLEN-wide CSR, `misa`, specifying the supported matrix arithmetic feature of the current hardware implementation.

Bits	Feature	Support Type
31	mfew	optional
30	mhp	optional
29	mfic	optional
28	miew	optional
27	mew64b	optional
26	mew32b	optional
25	mew16b	optional
24	mew8b	optional
23	mew4b	optional
22:19	0	reserved
18	mmmxf8mxf4	optional
17	mmmxf8	optional
16	mmmxf4	optional
15	mmf4f32	optional
14	mmf4bf16	optional
13	mmf4f16	optional
12	mmf8bf16	compulsory
11	mmbf20f32	optional
10	mmbf16f32	compulsory
9	mmf8f32	compulsory
8	mmf8f16	compulsory
7	mmf32f64	optional
6	mmf16f32	compulsory
5	mmf64f64	optional
4	mmf32f32	optional

Bits	Feature	Support Type
3	mmf16f16	compulsory
2	reserved	reserved
1	mmi8i32	compulsory
0	mmi4i32	optional

For each field, a value 0 means the corresponding feature is not supported, a value 1 means the corresponding feature is supported.

For `mmi4i32` field, a value 1 means the corresponding int4/uint4 oct-widen matrix multiplication instructions are supported. The source operands type of this field are (su)int4 and the destination operand is int32. This field is optional for hardware implementation.

For `mmi8i32` field, a value 1 means the corresponding int8/uint8 quad-widen matrix multiplication instructions are supported. The source operands type of this field are (su)int8 and the destination operand is int32. This field is compulsory for hardware implementation.

For `mmf16f16` field, a value 1 means the corresponding fp16 no-widen matrix multiplication instructions are supported. The source operands type of this field are fp16 and the destination operand is fp16. This field is compulsory for hardware implementation.

For `mmf32f32` field, a value 1 means the corresponding fp32 no-widen matrix multiplication instructions are supported. The source operands type of this field are fp32 and the destination operand is fp32. This field is optional for hardware implementation.

For `mmf64f64` field, a value 1 means the corresponding fp64 no-widen matrix multiplication instructions are supported. The source operands type of this field are fp64 and the destination operand is fp64. This field is optional for hardware implementation.

For `mmf16f32` field, a value 1 means the corresponding fp16 double-widen matrix multiplication instructions are supported. The source operands type of this field are fp16 and the destination operand is fp32. This field is compulsory for hardware implementation.

For `mmf32f64` field, a value 1 means the corresponding fp32 double-widen matrix multiplication instructions are supported. The source operands type of this field are fp32 and the destination operand is fp64. This field is optional for hardware implementation.

For `mmf8f16` field, a value 1 means the corresponding fp8 double-widen matrix multiplication instructions are supported. The source operands type of this field are fp8(E4M3 and E5M2) and the destination operand is fp16. This field is compulsory for hardware implementation.

For `mmf8f32` field, a value 1 means the corresponding fp32 quad-widen matrix multiplication instructions are supported. The source operands type of this field are fp8 and the destination operand is fp32. This field is compulsory for hardware implementation.

For mmbf16f32 field, a value 1 means the corresponding bf16 double-widen matrix multiplication instructions are supported. The source operands type of this field are bf16 and the destination operand is fp32. This field is compulsory for hardware implementation.

For mmbf20f32 field, a value 1 means the corresponding bf20 no-widen matrix multiplication instructions are supported. The source operands type of this field are bf20 and the destination operand is fp32. This field is optional for hardware implementation.

For mmf8bf16 field, a value 1 means the corresponding fp8 double-widen matrix multiplication instructions are supported. The source operands type of this field are fp8(E4M3 and E5M2) and the destination operand is bf16. This field is compulsory for hardware implementation.

For mmf4f16 field, a value 1 means the corresponding fp4 quad-widen matrix multiplication instructions are supported. The source operands type of this field are fp4 and the destination operand is fp16. This field is optional for hardware implementation.

For mmf4bf16 field, a value 1 means the corresponding fp4 quad-widen matrix multiplication instructions are supported. The source operands type of this field are fp4 and the destination operand is bf16. This field is optional for hardware implementation.

For mmf4f32 field, a value 1 means the corresponding fp32 oct-widen matrix multiplication instructions are supported. The source operands type of this field are fp4 and the destination operand is fp32. This field is optional for hardware implementation.

For mmmxf4 field, a value 1 means the corresponding mxfp4 matrix multiplication instructions are supported. The source operands type of this field are mxfp4 and the destination operand is fp32. This field is optional for hardware implementation.

For mmmxf8 field, a value 1 means the corresponding mxfp8 matrix multiplication instructions are supported. The source operands type of this field are mxfp8 and the destination operand is fp32. This field is optional for hardware implementation.

For mmmxf8mxf4 field, a value 1 means the corresponding mxfp8 and mxfp4 matrix multiplication instructions are supported. The source operands type of this field are mxfp8 and mxfp4 and the destination operand is fp32. This field is optional for hardware implementation.

For mew4b field, a value 1 means the 4bit-width related element wise instructions are supported. This field is optional for hardware implementation.

For mew8b field, a value 1 means the 8bit-width related element wise instructions are supported. This field is optional for hardware implementation.

For mew16b field, a value 1 means the 16bit-width related element wise instructions are supported. This field is optional for hardware implementation.

For mew32b field, a value 1 means the 32bit-width related element wise instructions are supported. This field is optional for hardware implementation.

For `mew64b` field, a value 1 means the 32bit-width related element wise instructions are supported. This field is optional for hardware implementation.

For `miew` field, a value 1 means integer element wise extension(including arithmetic instruction and conversion instructions) is supported. This field is optional for hardware implementation.

For `mfic` field, a value 1 means the element wise conversion between integer and float point extension is supported. This field is optional for hardware implementation.

For `mhp` field, a value 1 means the corresponding hybrid-precision matrix multiplication instructions are supported.

For `mfew` field, a value 1 means float point element wise extension(including arithmetic instruction and conversion instructions) is supported, including the . This field is optional for hardware implementation.

The `miew`, `mfew` and `mfic`, together with `mew4b`, `mew8b`, `mew16b`, `mew32b` and `mew64b`, jointly determine which element-wise instructions are supported.

3.2.1. Matrix Multiply Extensions

The below table describe the instructions included in different matrix multiply extensions.

Table 1. The Corresponding Matrix Multiply Instructions of Different MISA Bit

extension name	misa bit	instruction
xtheadmmi4bi32b	mmi4i32	th.mmacc.w.p
		th.mmaccu.w.p
		th.mmaccus.w.p
		th.mmaccsu.w.p
xtheadmmi8bi32b	mmi8i32	th.mmacc.w.b
		th.mmaccu.w.b
		th.mmaccus.w.b
		th.mmaccsu.w.b
xtheadmmf16bf16b	mmf16f16	th.mfmacc.h
xtheadmmf32bf32b	mmf32f32	th.mfmacc.s
xtheadmmf64bf64b	mmf64f64	th.mfmacc.d
xtheadmmf16bf32b	mmf16f32	th.mfmacc.s.h
xtheadmmf32bf64b	mmf32f64	th.mfmacc.d.s

xtheadmmf8bf16b	mmf8f16	th.mfmacc.h.e5m2
		th.mfmacc.h.e4m3
xtheadmmf8bf32b	mmf8f32	th.mfmacc.s.e5m2
		th.mfmacc.s.e4m3
xtheadmmbf16bf32b	mmbf16f32	th.mfmacc.s.bf16
xtheadmmbf20bf32b	mmbf20f32	th.mfmacc.s.bf20
xtheadmmf8bbf16b	mmf8bf16	th.mfmacc.bf16.e5m2
		th.mfmacc.bf16.e4m3
xtheadmmf4bf16b	mmf4f16	th.mfmacc.h.e2m1
xtheadmmf4bbf16b	mmf4bf16	th.mfmacc.bf16.e2m1
xtheadmmf4bf32b	mmf4f32	th.mfmacc.s.e2m1
xtheadmmmxf4b	mmmxf4	th.mfmacc.s.mxe2m1
		th.mfmacc.s.mxe2m1.ue4m3
xtheadmmmxf8b	mmmxf8	th.mfmacc.s.mxe5m2
		th.mfmacc.s.mxe4m3
xtheadmmmxf8bmxf4b	mmmxf8mxf4	th.mfmacc.s.mxe5m2mxe2m1
		th.mfmacc.s.mxe4m3mxe2m1

xtheadmhp	mhp	th.mmacc.w.bp
		th.mmaccu.w.bp
		th.mmaccus.w.bp
		th.mmaccsu.w.bp
		th.mfmacc.h.hb
		th.mfmacc.h.hp
		th.mfmaccu.h.hb
		th.mfmaccu.h.hp
		th.mfmacc.s.hb
		th.mfmacc.s.hp
		th.mfmaccu.s.hb
		th.mfmaccu.s.hp
		th.mfmacc.h.e4m3e2m1
		th.mfmacc.h.e5m2e2m1
		th.mfmacc.s.e4m3e2m1
		th.mfmacc.s.e5m2e2m1

3.2.2. Element-Wise Instructions

This section describes the supported element-wise instructions when the different misa bit including `miew`, `mfew`, `mfic`, `mew4b`, `mew8b`, `mew16b`, `mew32b` and `mew64b` are set or not.

For integer arithmetic element-wise instructions, if the `miew` bit is 0, all integer arithmetic element-wise instructions are illegal.

- When the `miew` and `mew32b` are both set to 1, the integer arithmetic element-wise instructions with 32b-width operands are legal.

For float-point arithmetic element-wise instructions, if the `mfew` bit is 0, all float-point arithmetic element-wise instructions are illegal.

- When the `mfew` and `mew16b` are both set to 1, the float-point arithmetic element-wise instructions with 16b-width operands are legal.
- When the `mfew` and `mew32b` are both set to 1, the float-point arithmetic element-wise instructions with 32b-width operands are legal.
- When the `mfew` and `mew64b` are both set to 1, the float-point arithmetic element-wise

instructions with 64b-width operands are legal.

For integer/fixed-point conversion element-wise instructions, if the `miew` bit is 0, all integer/fixed-point conversion element-wise instructions are illegal.

- When the `miew`, `mew4b` and `mew8b` are all set to 1, the integer conversion element-wise instructions from int4 to int8 are legal.
- When the `miew`, `mew8b` and `mew32b` are all set to 1, the fixed-point conversion element-wise instructions from int32 to int8 are legal.

For float-point conversion element-wise instructions, if the `mfew` bit is 0, all float-point conversion element-wise instructions are illegal.

- When the `mfew`, `mew4b` and `mew8b` are all set to 1, the float-point conversion element-wise instructions between fp4 and fp8(e4m3/e5m2) are legal.
- When the `mfew`, `mew8b` and `mew16b` are all set to 1, the float-point conversion element-wise instructions between fp8(e4m3/e5m2) and fp16 are legal.
- When the `mfew`, `mew16b` and `mew32b` are all set to 1, the float-point conversion element-wise instructions between fp16/bf16 and fp32 are legal.
- When the `mfew`, `mew64b` and `mew32b` are all set to 1, the float-point conversion element-wise instructions between fp32 and fp64 are legal.
- When the `mfew`, `mew8b` and `mew32b` are all set to 1, the float-point conversion element-wise instructions from fp32 to fp8(e4m3/e5m2) are legal.

For float-point integer conversion element-wise instructions, if the `mfic` bit is 0, all float-point integer conversion element-wise instructions are illegal.

- When the `mfic`, `mew4b` and `mew8b` are all set to 1, the float-point integer conversion element-wise instructions from int4 to fp8(e4m3/e5m2) are legal.
- When the `mfic`, `mew8b` and `mew16b` are all set to 1, the float-point integer conversion element-wise instructions between int8 and fp16 are legal.
- When the `mfic` and `mew32b` are both set to 1, the float-point integer conversion element-wise instructions between int32 and fp32 are legal.

3.3. Matrix Start Row Register, xmrstart

The xmrstart read-write register indicates the first matrix row index to be executed by a matrix load/store instruction. Normally xmrstart is only written by hardware on a trap of matrix load/store instructions, the unsigned value of register specifies the row at which the execution should resume after a resumable trap is handled.

All matrix instructions, including mcfg/mcfgi, reset the xmrstart CSR to zero.

The xmrstart CSR is defined to have only enough writable bits to hold the largest row index(one less than the max row) or log2(RLEN/32). The upper bits of the xmrstart CSR are hardwired to zero(reads zero, writes ignored). For example, xmrstart would have 2 bits to represent row indices from 0 through 3.

3.4. Matrix Size Register, xmsize

Matrix size register is a XLEN-bit WARL read-write register. It can be updated by matrix configuration instructions. The matrix size register has three fields, sizeK, sizeN and sizeM. Bits[XLEN-1:32] are reserved.

bits	Name	Description
XLEN-1:XLEN-32	0	reserved if non-zero
31:16	sizeK[15:0]	column of Matrix A or Matrix B, in bytes
15:8	sizeN[7:0]	row of Matrix B
7:0	sizeM[7:0]	row of Matrix A

The sizeM/sizeN/sizeK field hold an unsigned integer specifying the source elements needed and the destination elements updated by a matrix instructions. The sizeK which is not the multiples of element size in byte will raise an illegal instruction exception.

For matrix-multiplication instructions, which computing C[M][N] += A[M][K]*BT[N][K], there are 3 source operands and 1 destination operand. Only sizeM x sizeN elements will be updated, the other elements are set by zeros. The source operands dimensions are defined as follows:

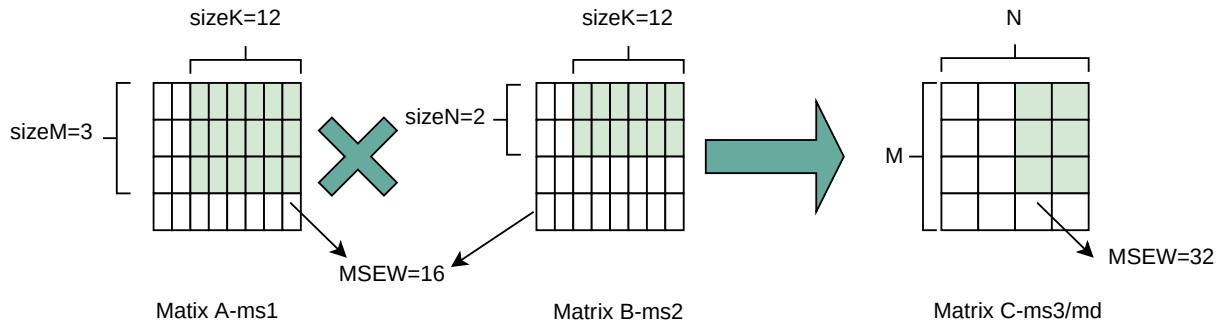
1. Matrix A: sizeM x (sizeK x 8 / MSEW)
2. Matrix B: sizeN x (sizeK x 8 / MSEW)
3. Matrix C: sizeM x sizeN·

Thus, there are the limitations of Matrix shape due to the matrix register.

1. sizeK ≤ xrlenb
2. sizeM ≤ RLEN/32
3. sizeN ≤ RLEN/32, for mfmacc.h.x sizeN ≤ 2*(RLEN/32)

When the result operand width is 16-bit, the matrix B are stored in two matrix register, which can fully use the destination matrix register.

The following picture shows an example of matrix register shapes for matrix-multiplication instruction. The RLEN of matrix registers is 128-bit, the source operand is 16-bit, the destination operand is 32-bit, and sizeM/sizeN/sizeK is 3/2/12.

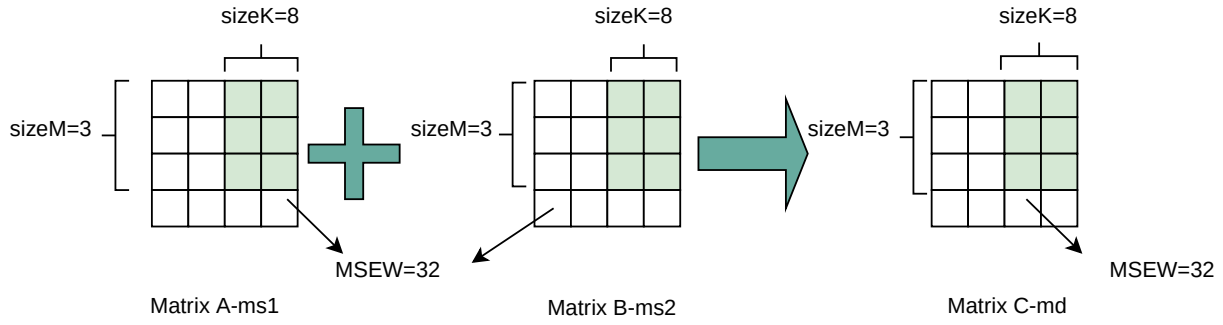


C += A * BT Matrix Multplication at sizeM/N/K(RLEN=128bit)

For element-wise and load/store instructions, the matrix shapes keep during the execution, which are specified by sizeM and sizeK. Only sizeM x sizeK elements will be updated, the other elements are set by zeros. The size limitations are:

1. sizeM ≤ RLEN/32
2. sizeK ≤ xrlenb

```
Taking int32 matrix add as an example , the configuration of  sizeM=3/sizeK=8
indicated MatrixA(3x2) + MatrixB(3x2) = MatrixC(3x2), only the green block
elements are used or updated by the instruction.
```



Element-wise Add Instructions at sizeM/K(RLEN=128bit)

3.5. Microscaling Format Descriptor Register, xmx(a/b)desc

The 'xmxadesc' and 'xmxbdesc' CSR are two WARL read-write register which are used to describe the microscaling format information for matrix A and matrix B. The 'xmxadesc' and 'xmxbdesc' have the same fields, 'colidx' and 'blksize'. Bits[XLEN-1:9] are reserved and should be written with zero.

Table 2. `xmx(a/b)desc` register layout

Bits	Name	Description
XLEN-1:18	0	Reserved if non-zero.
17:2	colidx	column index for the scale elements in the scale matrix register
1:0	blksize	blocksize of private elements for a scale element

Matrix A and matrix B have different microscaling format descriptor csr, which means they can have different blocksize and column index.

Only low log2(RLEN/SSEW) bits of colidx[15:0] are used.

The blksize field(xmx(a/b)desc[1:0]) represents four different blocksize.

Table 3. actual blocksize for different blksize configuration

xmx(a/b)desc[1:0]	blocksize
2’b00	16
2’b01	32
2’b10	64
2’b11	128

3.6. Matrix Control and Status Register, xmcsr

The `xmcsr` CSR is a WARL read-write register. Bits[XLEN-1:12] are reserved and should be written with zero. The `mcsr` register has 5 fields. The xmxrm, xmsat, xmfflags, xmfrm, xmsaten separate CSRs can also be accessed via fields in the matrix control and status CSR, xmcsr.

Table 4. `xmcsr` register layout

Bits	Name	Description
XLEN-1:12	0	Reserved if non-zero.
11	xmsaten	integer matrix multiplication and fp8 saturation mode.
10:8	xmfrm	Float-point arithmetic instruction rounding mode.
7:3	xmfflags	Float-point arithmetic instruction accrued exception flags
2	xmsat	Fixed-point arithmetic instruction accrued saturation flag.
1:0	xmxrm	Fixed-point arithmetic instruction rounding mode.

3.7. Matrix Fixed-Point Rounding Mode, xmxrm

`xmxrm` field indicates the rounding mode of fixed-point instructions. Suppose the pre-rounding result is v, and d bits of that result are to be rounded off. Then the rounded result is (v >> d) + r, where r depends on the rounding mode as specified in the following table.

mxrm	Mnemonic	meaning	Rounding Increment r
00	RNU	round-to-nearest-up (add +0.5 LSB)	v[d-1]
01	RNE	round-to-nearest-even	v[d-1] & (v[d-2:0] != 0 | v[d])
10	RDN	round-down (truncate)	0
11	ROD	round-to-odd (OR bits into LSB)	!v[d] & v[d-1:0] != 0

The rounding functions are used to represent this operation in the instruction descriptions below:

```
roundoff_unsigned(v, d) = (unsigned(v) >> d) + r
roundoff_signed(v, d)   = (signed(v) >> d) + r
```

3.8. Matrix Fixed-Point Saturation Flag, xmxsat

The matrix fixed-point saturation flag `xmxsat` holds a single read-write least-significant bit(xmxsat[0]) that indicates if a fixed-point instruction has had to saturate an output value to fit into a destination format. The upper bits, xmxsat[XLEN-1:1] , should be written as zeros. The matrix fixed-point saturation flag(xmxsat) is given a separate CSR address to allow independent access, but is also reflected as a field in xmcsr.

3.9. Matrix Float-Point Exception Flags, xmfflags

The matrix float-point exception flags(xmfflags) holds a five-bit read-write fields in the least-significant bits(xmfflags[4:0]). The upper bits, xmfflags[XLEN-1:5] , should be written as zeros. The matrix float-point exception flags(xmfflags) is given a separate CSR address to allow independent access, but is also reflected as a field in xmcsr. The float matrix multiplication and float pointwise instructions update accrued exception fflags in xmfflags.

bits	Meaning
0	NX
1	UF
2	OF
3	reserved for future use

bits	Meaning
4	NV

3.10. Matrix Float-Point Rounding Mode, xmfrm

The matrix float-point rounding mode`xmfrm` holds a three-bit read-write fiels in the least-significant bits(xmfrm[2:0]). The upper bits, xmfrm[XLEN-1:3] , should be written as zeros. The matrix float-point rounding mode `xmfrm` is given a separate CSR address to allow independent access, but is also reflected as a field in xmcsr.

mfrm	Mnemonic	Meaning
000	RNE	Round to Nearest, ties to Even
001	RTZ	Round towards Zero
010	RDN	Round Down (towards −∞)
011	RUP	Round Up (towards +∞)
100	RMM	Round to Nearest, ties to Max Magnitude
101		Invalid
110		Invalid
111		Invalid

If `xmfrm` is set to an invalid value (101-111), any subsequent attempt to execute a floating-point operation with a dynamic rounding mode will cause an illegal instruction exception.

3.11. Matrix Saturation Mode Enable, xmsaten

The matrix saturation mode `xmsaten` holds a single read-write least-significant bit(xmsaten[0]) that defines the saturation mode when the operation result is fp8 or integer. The upper bits, xmsaten[XLEN-1:1] , should be written as zeros. The matrix saturation mode(xmsaten) is given a separate CSR address to allow independent access, but is also reflected as a field in xmcsr.

msaten	Meaning
0	non-saturation mode
1	saturation mode

Conversion from all other values first applies rounding to reduce the mantissa bit count to that of the destination FP8 format. After that, if the rounded magnitude is above the maximum destination magnitude:

```
if `msaten` = 1:
    fp8 conversions: generate the max normal FP8 magnitude
    mmacc operations: the result should be saturated
if `msaten` = 0:
    fp8 conversion:E4M3 destination: generate a NaN
                   E5M2 destination: generate an infinity
    mmacc operations: ignore the overflow and wrap around the result
```

3.12. Matrix Context Status in mstatus and sstatus

A 2-bit matrix context status field, MS, should be added to mstatus and shadowed in sstatus. It is defined analogously to the vector context status field, VS.

ms[1:0]	Meaning
00	All Off
01	Initial
10	Clean
11	Dirty

Attempts to execute any matrix instructions, or to access the matrix CSRs raise an illegal instruction exception when MS is set to off. If MS is set to initial or clean, executing any instructions that change the matrix state will change the ms to dirty.

An implementation can use the activity of the Initial state to influence the choice of power-saving states.

Chapter 4. Instruction Format

The 32-bit matrix extension instruction format fit under the custom-1(0101011) major opcode.

The md/ms1/ms2/ms3 fields are 3-bit width to index the eight matrix registers.

md/ms1/ms2/ms3[2:0]	Matrix Register Index
000	m0
001	m1
010	m2
011	m3
100	m4
101	m5
110	m6
111	m7

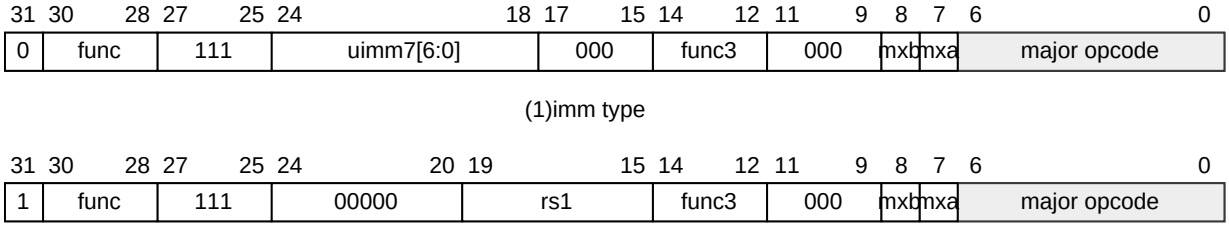
Generally, the size field at [11:10]-bit indicate the source operand bit-width.

size[1:0]	source operand width
00	8bit
01	16bit
10	32bit
11	64bit

There are five main types of instruction formats, as illustrated below.

4.1. Configuration instructions

The following figure shows the encoding format for the configuration instructions.



Configuration Instructions

The func3 field of the configuration instructions is '000'.

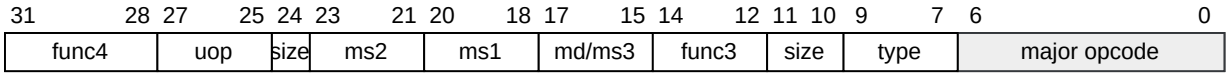
There exists two main types of configuration instructions. One is the imm-type, whose source data is the immediate number encoded in the instruction. Another is the register-type, whose source data is in the 'rs1' register. The two type configuration instructions are distinguished by [31]-bit. The func field at [30:28]-bit defines the different configuration instructions.

For the mcfg<m/n/k>(i) instructions, the [11:7]-bit is '00000'. For the mxcfg(i) instructions, the [11:9]-bit is '000', and the mxa/mxb fields indicate whether config the xmxadesc/xmxbdesc or not. If the mxa/mxb is 1, the the mxcfg(i) instructions need change the xmxadesc/xmxbdesc csr.

For the mrelease instruction, the [31]-bit is '0', and the func field at [30-28]-bit is '111'. The other fields are all 0.

4.2. Matrix Multiply instructions

The following figure shows the encoding format for the matrix multiply instructions.



Matrix Multiplication Instruction

The func3 field of the matrix multiply instructions is '000'. The uop field at [27:25]-bit is always '000'. The size field at [11:10]-bit indicates the source operand bit-width.

When the func4 field at [31:28]-bit is '0001', the instructions are float point matrix multiply instructions. If the size field at [24]-bit is 0, the source operand width equals to the destination operand width. If the size field at [24]-bit is 1, the source operand width widens to the destination operand width. The type field at [9:7]-bit indicates the source operand are double-widened or quad-widened, and indicates the operand type, like fp16, bf16, e4m3, e5m2.

When the func4 field at [31:28]-bit is '0010', the instructions are integer matrix multiply instructions. The type field at [9:7]-bit indicates the two source operands are signed or unsigned. If the Theadmint4 extension is supported, the size field at [11:10]-bit is '00', and the size field at [24]-bit is '1'. If the Theadmhp extension is supported, the A8W4 integer matrix multiply instructions are represented by setting the size field at [11:10]-bit to '10', and the size field at [24]-bit indexes the high

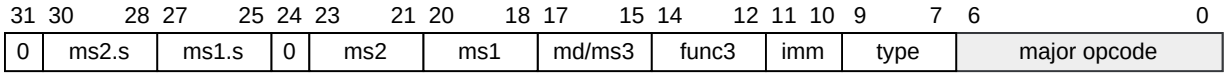
or low half part of ms2 matrix register.

When the func4 field at [31:28]-bit is '0011', the instructions are hybrid-precision float point matrix multiply instructions. The [9]-bit is used to indicate the source operands are float-point/integer or float-point/float-point. The [24]-bit is used to indicate the integer source is signed or unsigned. If the [11]-bit is '0', the destination operand type is fp16. If the [11]-bit is '1', the destination operand type is fp32. For float-point/integer hybrid-precision instructions, if the [10]-bit is '0', the instruction type is A16W8, and the [7]-bit are used to index the high or low half part of ms2 matrix register when the result is fp32, if the [10]-bit is '1', the instruction type is A16W4, and the [8:7]-bit or the [7]-bit are used to index a quarter or a half part of ms2 matrix register. For float-point/float-point hybrid-precision instructions, if the [10]-bit is '0', the instruction type is E4M3*E2m1, and the [7]-bit are used to index the high or low half part of ms2 matrix register when the result is fp32, if the [10]-bit is '1', the instruction type is E5M2*E2M1, and the [7]-bit are used to index a half part of ms2 matrix register when the result is fp32.

When the func4 field at [31:28]-bit is '0100', the instructions are Theadmfp4 extension instructions. The type field at [9:7]-bit indicates the destination operand type, including fp16, bf16, fp32.

4.3. Microscaling Matrix Multiply instructions

The following figure shows the encoding format for the microscaling matrix multiply instructions.

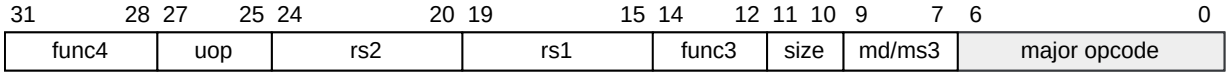


Microscaling Matrix Multiplication Instruction

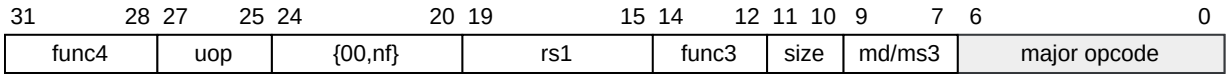
The func3 field of the microscaling matrix multiply instructions is '010'. The type field at [9:7]-bit indicates different source operand type for microscaling matrix multiply instructions. The [10]-bit indexes the high or low half part of ms2 matrix register for mxfp8-mxfp4 hybrid microscaling matrix multiply instructions.

4.4. Load and Store instructions

The following figure shows the encoding format for the load and store instructions.



load & strore instructions



whole register load & strore instruction

The func3 field of the matrix multiply instructions is '000'. When the uop field at [27:25]-bit is '100', the instructions are load instructions. When the uop field at [27:25]-bit is '101', the instructions are

store instructions. The size field at [11:10]-bit indicates the operand bit-width in load or store operations.

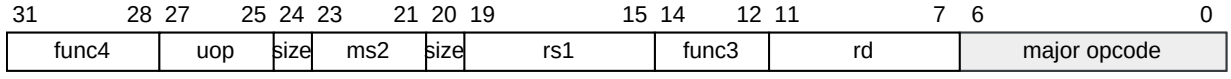
When the func4 field at [31:28]-bit is '0000', the instructions are normal load or store instructions. When the func4 field at [31:28]-bit is '0001', the instructions are stream load or store instructions. When the func4 field at [31:28]-bit is '0010', the instructions are whole register load or store instructions. When the func4 field at [31:28]-bit is '0011', the instructions are transposed load or store instructions. When the func4 field at [31:28]-bit is '0100', the instructions are transposed stream load or store instructions. When the func4 field at [31:28]-bit is '0101', the instructions are prefetch load instructions. When the func4 field at [31:28]-bit is '0110', the instructions are transposed prefetch load instructions.

For whole register load or store instructions, the nf field at [22:20]-bit indicates the register number.

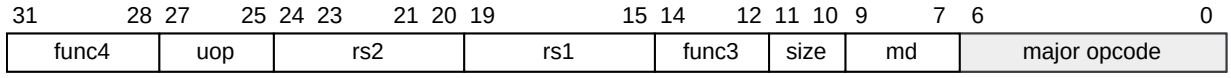
nf[2:0]	**register number**
000	1
001	2
011	4
111	8
others	reserved

4.5. MISC instructions

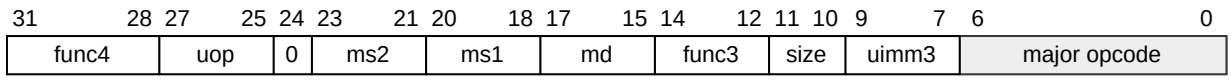
The following figure shows the encoding format for the misc instructions.



(1)mmov<b/h/w/d>.x.m



(2)mmov<b/h/w/d>.m.x & mdup<b/h/w/d>.m.x



(3)others

The func3 field of the misc instructions is '000'. The uop field of the misc instructions is '110', except mzero, mmov.mm, and mmov.mv.i.

For the mzero instruction, the uop field is '000', and the func4 field is '1010'. The uimm3 field at

[9:7]-bit indicates how many registers are set to 0. The ms1 and ms2 field are all 0.

For the mmov.mm instruction, the uop field is '000', the func4 field is '0000', the size field at [11:10]-bit is '00', and the uimm3 field at [9:7]-bit is 001. The ms2 field is '000'.

For the mmov.mv.i instruction, the uop field is '010', the func4 field is '0000', and the size field at [11:10]-bit is '00'. The uimm3 field at [9:7]-bit indicates the selected row number of ms1. The ms2 field is '000'.

For the mmov<b/h/w/d>.x.m instructions, the uop field is '110', and the func4 field is '0000'. The [24]-bit and the [20]-bit indicate the element size for the matrix-to-scaler move operations.

For the mdup<b/h/w/d>.m.x instructions, the uop field is '110', the func4 field is '0001', and the rs1 field is all 0. The size field at [11:10]-bit indicate the element size for the matrix-to-scaler duplicate operations.

For the mmov<b/h/w/d>.m.x instructions, the uop field is '110', and the func4 field is '0010'. The size field at [11:10]-bit indicate the element size for the matrix-to-scaler move operations.

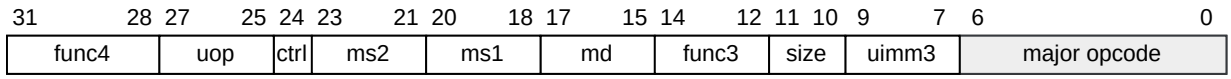
For the mpack/mpackhl/mpackhh instructions, the uop field is '110', the func4 field is '0011', and the size field at [11:10]-bit is '00'. The uimm3 field at [9:7]-bit indicates the three pack types.

For the mrslide and mcslide instructions, the uop field is '110', and the func4 field is '0100'. The size field at [11:10]-bit is '00' for the mrsildedown and mrslideup instructions. The size field at [11:10]-bit indicates the element size for the mcslide instructions. The uimm3 field at [9:7]-bit indicates the row index or the column index. The ms2 field indicates the different instruction types for mrslide and mcslide instructions.

For the mcmov<b/h/w/d> instructions, the uop field is '110', and the func4 field is '0101'. The ms2 field is '000'. The size field at [11:10]-bit indicates the element size for the mcmov<b/h/w/d> instructions. The uimm3 field at [9:7]-bit indicates the column index.

4.6. Element-wise instructions

The following figure shows the encoding format for the element-wise instructions.



Element-wise instructions

The func3 field of the element-wise instructions is '001'.

For the integer element-wise arithmetic instructions, the uop field is '000' and '010'. When the uop field is '000', the two operands are all matrix. When the uop field is '010', one operand is a matrix, and one operand is a row in a matrix, which is indexed by the uimm3 filed. The size field at [11:10]-bit indicates the element size. The func4 field and the ctrl field indicate different integer operation type.

For the fixed-point convert instructions, the uop field is '000' and '010'. When the uop field is '000', the two operands are all matrix. When the uop field is '010', one operand is a matrix, and one operand is a row in a matrix, which is indexed by the uimm3 filed. The size field at [11:10]-bit indicates the element size. When the func4 field is '0110', the source operand is signed, when the func4 field is '0111', the source operand is unsigned. The ctrl field at [24]-bit index the first or second quarter for the destination result.

If the Theadmint4 extension is supported, the int4 and int8 convert instructions are supported. For the integer-integer convert instructions, the func4 field is '1010', the ctrl field is '1', the ms2 field is '000', and the size field is '00'. The uop field is '000' and '010', which are used to index the low half or high half of matrix register. The uimm3 field is '000' and '001', which are used to distinguish the signed and unsigned operand.

For the float point element-wise arithmetic instructions, the uop field is '001' and '011'. When the uop field is '001', the two operands are all matrix. When the uop field is '011', one operand is a matrix, and one operand is a row in a matrix, which is indexed by the uimm3 filed. The ctrl field is '0'. The size field at [11:10]-bit indicates the element size. The func4 field indicate different float point operation type.

For the float point convert instructions, the func4 field is '0101', and the ms2 field is '000'. The size field indicates the source operand width. When the ctrl field is '0', the convert type is narrow-convert. When the ctrl field is '1', the convert type is widen-convert. The uop field is '001' and '011', which indexes the high or low half of the related matrix register. For fp32 to fp8 conversion, the uop field is '001' and '011', which indexes the first or second quarter of the related matrix register. The uimm3 field is used to distinguish the different data type, like fp16 and bf16, e4m3 and e5m2.

If the Theadmfp4 extension is supported, the float point convert instructions between fp8 and fp4 are supported. The func4 field is '0101', the ms2 field is '000', and the size field is '00'. When the ctrl field is '0', the convert type is narrow-convert. When the ctrl field is '1', the convert type is widen-convert. The uop field is '101' and '111', which indexes the high or low half the related matrix register.

For the float-point integer convert instructions, the func4 field is '0110'. When the convert is int4 to fp8, the ctrl field is '1', otherwise, the ctrl field is '0'. When the convert is int4 to fp8, the uimm3 field is used to distinguish the e4m3/e5m2 destination format, otherwise, the uimm3 field is '0'. The uop field is '001' and '011', which indexes the high or low half of the related matrix register. The size field indicates the source operand width. The ms2 field are used to distinguish different operand types, like signed and unsigned, float-point and integer, widen or no-widen.

Chapter 5. Instructions

5.1. Configuration Instructions

5.1.1. Matrix Size Configuration Instructions

Due to hardware resource constraints, one of the common ways to handle large-sized matrix multiplication is "tiling", where each iteration of the loop processes a subset of elements, and then continues to iterate until all elements are processed. The Matrix extension provides direct, portable support for this approach.

The block processing of matrix multiplication requires three levels of loops to iterate in the direction of m/k/n of the input application tiles.

For each iteration, most of the matrix multiplication can fully use the matrix registers. But when dealing with the tail of matrix multiplication, the length of each multiplication dimension like sizeM/sizeN/sizeK need to be updated to proper tail size.

Matrix configuration instructions configure a field or the whole matrix size configuration register. The field retains the value if not changed by a configuration instruction. The index field of the instruction indicates which field is updated, sizeM/sizeK/sizeN or the entire configure register as following table shows.

instruction	effect on matrix size
mcfgm(i)	msize.byte0 = x[rs1]
mcfgn(i)	msize.byte1 = x[rs1]
mcfgk(i)	msize.half1 = x[rs1]
mcfg	msize.byte0 = x[rs1] .byte0
	msize.byte1 = x[rs1] .byte1
	msize.half1 = x[rs1] .half1

```
#imm type
th.mcfg<m/n/k>i   uimm7   # uimm7 = new sizeM/N/K

#register type
th.mcfg<m/n/k>     rs1    # rs1[7:0] = new sizeM/N  or  rs1[15:0] = new sizeK

#entire register
th.mcfg  rs1       # rs1 = new xmsize
```

5.1.2. Mrelease Instruction

Mrelease instruction sets `MS` field in `mstatus` register to 01 (initial). This instruction is often used after the executing of a set of matrix instructions, indicating that the data in matrix registers can be discarded. Mrelease instruction can reduce context switch costs.

```
th.mrelease
```

5.2. Matrix Multiplication Instructions

Matrix multiplication instructions take matrix A(sizeMxsizeK) and matrix BT(sizeNxsizeK) from matrix registers specified by ms1 and ms2, and accumulate the multiplication result of A[M][K] * (BT[N][K]) to matrix C(sizeM x sizeN) from md register, the output will overwrite the accumulation register.

1. shape of matrix A: M rows(sizeM), K columns(sizeK x 8 / MSEW)
2. shape of matrix B: N rows(sizeN), K columns(sizeK x 8 / MSEW)
3. shape of matrix C: M rows(sizeM), N columns(sizeN)

The function description of `C = A x BT` is as follows:

```
#A[i,k]  means the i row and k column element of ms1
#B[j,k]  means the j row and k columns element of ms2
#C[i,j]  means the i row and j column element of md

for(int i=0; i<sizeM; i++) {
  for(int j=0; j<sizeN; j++) {
      for(int k=0; k<(sizeK x 8 / MSEW); k++){
         C[i,j] += A[i,k] * B[j,k];
}}}
```

The ISA specification provides different instructions to support float and integer matrix multiplication operation. Hardware design has the flexibility of supported data types.

category	instructions	Source Operand Type	Accumulator Type
Float	mfmacc.h	fp16	fp16
Float	mfmacc.s	fp32	fp32
Float	mfmacc.d	fp64	fp64
Float	mfmacc.h.<e4m3/e5m2>	fp8(e4m3/e5m2)	fp16

category	instructions	Source Operand Type	Accumulator Type
Float	mfmacc.bf16.<e4m3/e5m2>	fp8(e4m3/e5m2)	bf16
Float	mfmacc.s.h	fp16	fp32
Float	mfmacc.s.bf16	bf16	fp32
Float	mfmacc.d.s	fp32	fp64
Float	mfmacc.s.<e4m3/e5m2>	fp8(e4m3/e5m2)	fp32
Int	mmacc.w.b	int8	int32
Int	mmaccu.w.b	uint8	int32
Int	mmaccsu.w.b	(su)int8	int32
Int	mmaccus.w.b	(us)int8	int32

The definition of fp8 aligns with the OCP 8-bit Floating Point Specification (OFP8). The canonical NaN for both E4M3 and E5M2 is 0x7f.

The mfmacc.x.y is float matrix multiplication operation, where x is destination operand width and y is source operand width. If x=y, y can be eliminated. The basic operation of float matrix multiplication is float dot-product, the float dot-product operations follow the IEEE-754/2008 standard. IEEE754/2008 standard's definition gives the implementation-specific flexibility of matrix-multiplication operation. The definitions are as followings:

1. Subnormal input/output: subnormal inputs and output are correctly computed and not treated as zero, tininess is detected after rounding.
2. The floating-point matrix-multiplication instructions which multiply single elements from each source operand and accumulate their product to each accumulator element. The reduction is done as an unordered-sum.Rounding are done after addition to accumulator.

The mmacc.x.y is int matrix multiplication operation, where x is destination operation width and y is source operation. If x=y, y can be eliminated. If `xmsaten` equals 1, the output should be saturated. Otherwise, the mmacc operations ignore the overflow and wrap around the result .

As integer matrix multiplication operation with non-widen or widen output is uncommon in AI scenarios, the matrix instruction set does not include such instructions by default.The hardware can extend the mmacc.<b/h/w/d> or mmacc.<h/w/d>.<b/h/w> instructions as needed.

```
# integer matrix multiplication and add, md = md + ms1 * ms2.
th.mmacc.w.b   md, ms2, ms1    #signed 8bit,  output quad-widen
th.mmaccu.w.b  md, ms2, ms1    #unsigned 8bit,  output quad-widen
th.mmaccus.w.b md, ms2, ms1    #unsigned-signed 8bit,  output quad-widen
```

```
th.mmaccsu.w.b md, ms2, ms1    #signed-unsigned 8bit,  output quad-widen

# Float point matrix multiplication and add, md = md + ms1 * ms2.
th.mfmacc.h        md, ms2, ms1  # 16-bit float point
th.mfmacc.s        md, ms2, ms1  # 32-bit float point
th.mfmacc.d        md, ms2, ms1  # 64-bit float point
th.mfmacc.h.e4m3     md, ms2, ms1  # 8-bit float point, output double-widen
th.mfmacc.h.e5m2     md, ms2, ms1  # 8-bit float point, output double-widen
th.mfmacc.bf16.e4m3  md, ms2, ms1  # 8-bit float point, output double-widen
th.mfmacc.bf16.e5m2  md, ms2, ms1  # 8-bit float point, output double-widen
th.mfmacc.s.h      md, ms2, ms1  # 16-bit float point, output double-widen
th.mfmacc.s.bf16   md, ms2, ms1  # 16-bit float point, output double-widen
th.mfmacc.d.s      md, ms2, ms1  # 32-bit float point, output double-widen
th.mfmacc.s.e4m3     md, ms2, ms1  # 8-bit float point, output quad-widen
th.mfmacc.s.e5m2     md, ms2, ms1  # 8-bit float point, output quad-widen
```

Hardware can support a subset of these instructions above. Executing of unsupported instructions will raise illegal instruction exception. The supported instructions are shown in `misa` registers.

The field `mfrm` in `mcsr` register indicates the rounding mode of float-point matrix instructions.

For executing matrix multiplication instructions, illegal instruction exception will occur under unsupported corresponding sizeM/N/K setting. The illegal condition is described in 3.3 section.

5.2.1. Float Matrix Multiplication(non-widen)

Non-widen float matrix multiplication indicates the source and destination operands data width keep the same which are encoded in the instruction.

- mfmacc.h: fp16 floating-point, illegal if 'mmf16f16' of xmisa register is 0
- mfmacc.s: fp32 floating-point, illegal if 'mmf32f32' of xmisa register is 0
- mfmacc.d: fp64 floating-point, illegal if 'mmf64f64' of xmisa register is 0

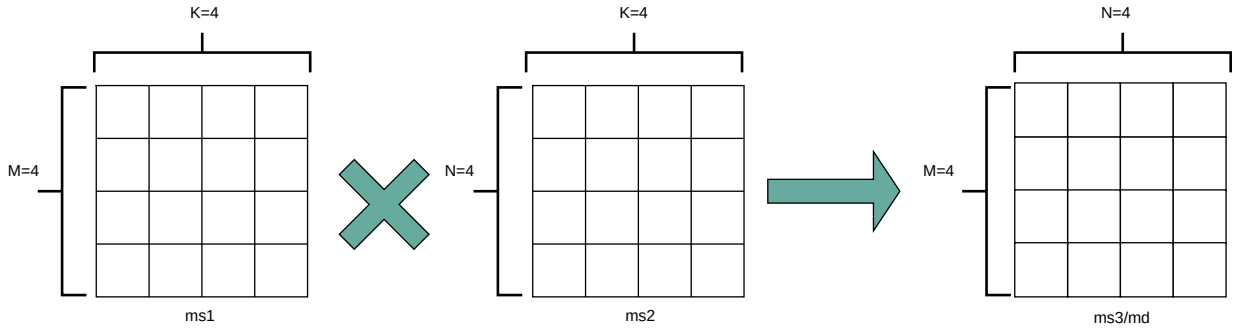
```
#float matrix multiplication, md = md + ms1*{ms2, ms2+1}
 mfmacc.h md, ms2, ms1

 #float matrix multiplication, md = md + ms1*ms2
 mfmacc.s md, ms2, ms1

 #float matrix multiplication, {md, md+1} = {md, md+1} + ms1*ms2
 mfmacc.d md, ms2, ms1
```

For mfmacc.s, the legal matrix shape is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/32

- matrixB: N ≤ RLEN/32, K ≤ RLEN/32
- matrixC: M ≤ RLEN/32, N ≤ RLEN/32

The operation of mfmacc.s is shown below for RLEN=128.

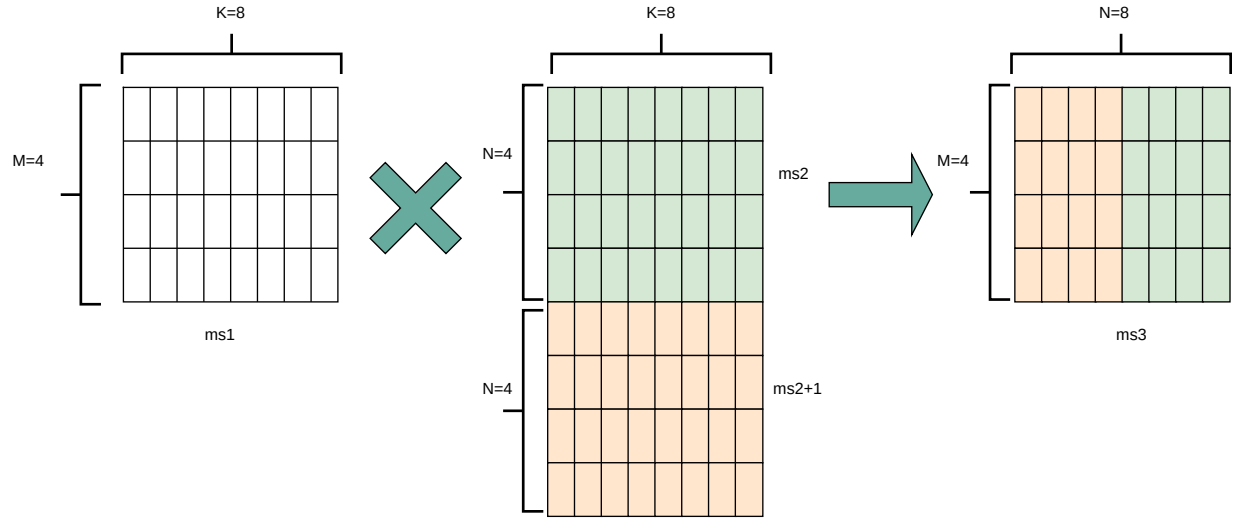


mfmacc.s RLEN=128bit

For mfmacc.h, 16-bit float matrix multiplication and add instruction, the element is 16-bit width. The max matrix shape is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/16
- matrixB: N ≤ RLEN/16, K ≤ RLEN/16
- matrixC: M ≤ RLEN/32, N ≤ RLEN/16

As data width for matrix B is twice that of matrix A and C, two matrix register(register-pair) are used by Matrix B specified by ms2 and ms2+1. Instructions specifying an odd-numbered ms2 is reserved. The operation of mfmacc.h is shown below for RLEN=128.

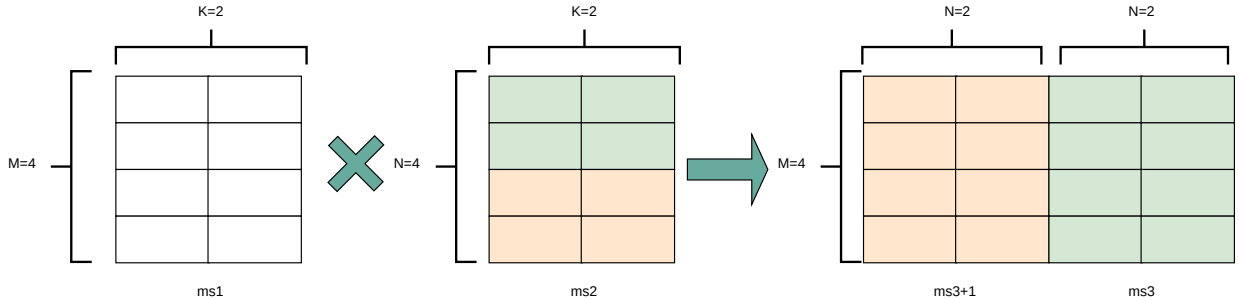


mfmacc.h RLEN=128bit

For mfmacc.d, 64-bit float matrix multiplication and add instruction, The maximum matrix shape is:

1. matrixA: M ≤ RLEN/32, K ≤ RLEN/64
2. matrixB: N ≤ RLEN/32, K ≤ RLEN/64
3. matrixC: M ≤ RLEN/32, N ≤ RLEN/32

As data width for matrix C is twice that of matrix A and B, two matrix register(register-pair) are used by Matrix C specified by md and md+1. Instructions specifying an odd-numbered md is reserved. The operation of mfmacc.d is shown below for RLEN=128.



mfmacc.d RLEN=128bit

Summary for max matrix size of non-widen mfmacc instructions for typical RLEN:

		matrix A			**matrix B**			**matrix C**		
	RLEN	M	K	matrix width	N	K	matrix width	M	N	matrix width
mfmacc.s	128	4	4	512 bits	4	4	512 bits	4	4	512 bits
	256	8	8	2048 bits	8	8	2048 bits	8	8	2048 bits
	512	16	16	8192 bits	16	16	8192 bits	16	16	8192 bits
mfmacc.h	128	4	8	512 bits	8	8	1024 bits	4	8	512 bits
	256	8	16	2048 bits	16	16	4096 bits	8	16	2048 bits
	512	16	32	8192 bits	32	32	16384 bits	16	32	8192 bits
mfmacc.d	128	4	2	512 bits	4	2	512 bits	4	4	1024 bits
	256	8	4	2048 bits	8	4	2048 bits	8	8	4096 bits
	512	16	8	8192 bits	16	8	8192 bits	16	16	16384 bits

5.2.2. Float Matrix Multiplication(double-widen)

Double-widen float matrix multiplication indicates destination operand data width is twice that of the source operand. The data width of source operand is in instruction encoding.

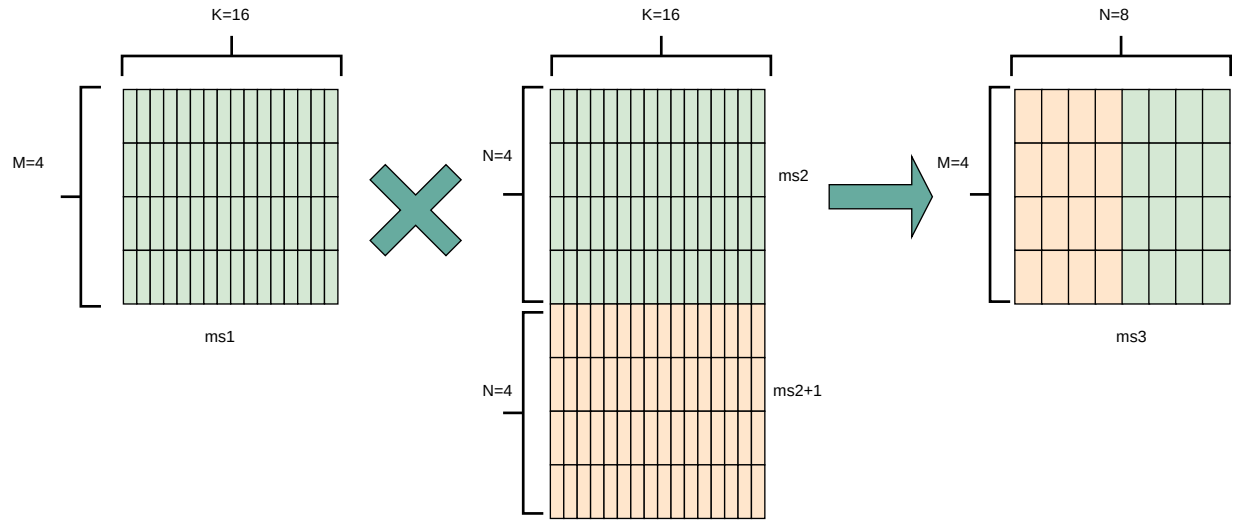
- mfmacc.h.<e4m3/e5m2>: fp8(e4m3/e5m2) floating-point source and fp16 result ,illegal if 'mmf8f16' of xmisa register is 0.
- mfmacc.bf16.<e4m3/e5m2>: fp8(e4m3/e5m2) floating-point source and bf16 result ,illegal if 'mmf8bf16' of xmisa register is 0.
- mfmacc.s.h: fp16 floating-point source and fp32 result ,illegal if 'mmf16f32' of xmisa register is 0.
- mfmacc.s.bf16: bf16 floating-point source and fp32 result ,illegal if 'mmbf16f32' of xmisa register is 0.
- mfmacc.d.s: fp32 floating-point source and fp64 result , illegal if 'mmf32f64' of xmisa register is 0.

```
#float matrix multiplication, output double_widen

#md = md + ms1*{ms2, ms2+1}
th.mfmacc.h.e4m3 md, ms2, ms1
th.mfmacc.h.e5m2 md, ms2, ms1
th.mfmacc.bf16.e4m3 md, ms2, ms1
th.mfmacc.bf16.e5m2 md, ms2, ms1

#md = md + ms1*ms2
th.mfmacc.s.bf16 md, ms2, ms1
th.mfmacc.s.h md, ms2, ms1

#{md, md+1} = {md, md+1} + ms1*ms2
th.mfmacc.d.s md, ms2, ms1
```

For mfmacc.<h/bf16>.<e5m2/e4m3> , the legel matrix shape is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/8
- matrixB: N ≤ RLEN/16, K ≤ RLEN/8
- matrixC: M ≤ RLEN/32, N ≤ RLEN/16

As data width for matrix B is twice that of matrix A and C, two matrix register(register-pair) are used by Matrix B specified by ms2 and ms2+1. Instructions specifying an odd-numbered ms2 is reserved. The operation of mfmacc.<h/bf16>.<e5m2/e4m3> is shown below for RLEN=128.

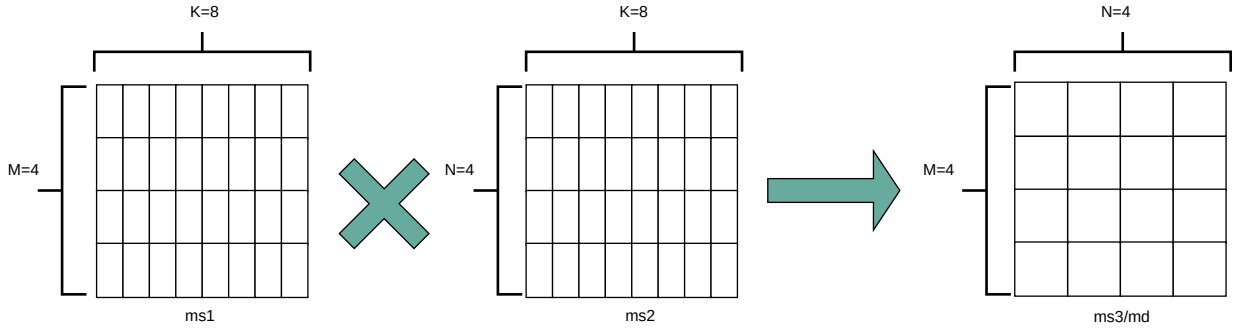


mfmacc.<h/bf16>.<e5/e4> RLEN=128bit

For mfmacc.s.h, the legel matrix shape is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/16
- matrixB: N ≤ RLEN/32, K ≤ RLEN/16
- matrixC: M ≤ RLEN/32, N ≤ RLEN/32

The operation of mfmacc.s.h is shown below for RLEN=128.

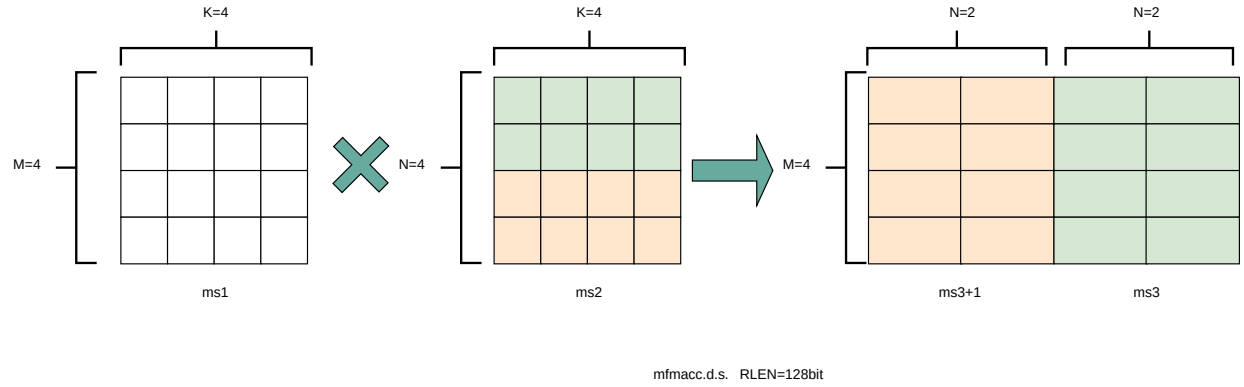


mfmacc.s.h RLEN=128bit

For mfmacc.d.s, the legal matrix shape is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/64
- matrixB: N ≤ RLEN/32, K ≤ RLEN/64
- matrixC: M ≤ RLEN/32, N ≤ RLEN/32

As data width for matrix C is twice that of matrix A and B, two matrix register(register-pair) are used by Matrix C specified by md and md+1. Instructions specifying an odd-numbered md is reserved.
The operation of mfmacc.d is shown below for RLEN=128.



Summary for max matrix size of double-widen mfmacc instructions for typical RLEN:

		matrix A			**matrix B**			**matrix C**		
	RLEN	M	K	matrix width	N	K	matrix width	M	N	matrix width
mfmacc.<h/bf16>.<e5m2/e4m3>	128	4	16	512 bits	8	16	1024 bits	4	8	512 bits
	256	8	32	2048 bits	16	32	4096 bits	8	16	2048 bits
	512	16	64	8192 bits	32	64	16384 bits	16	32	8192 bits
mfmacc.s.h	128	4	8	512 bits	4	8	512 bits	4	4	512 bits
	256	8	16	2048 bits	8	16	2048 bits	8	8	2048 bits
	512	16	32	8192 bits	16	32	8192 bits	16	16	8192 bits
mfmacc.d.s	128	4	4	512 bits	4	4	512 bits	4	4	1024 bits
	256	8	8	2048 bits	8	8	2048 bits	8	8	4096 bits
	512	16	16	8192 bits	16	16	8192 bits	16	16	16384 bits

5.2.3. Float Matrix Multiplication(quad-widen)

Quad-widen float matrix multiplication indicates destination operand data width is quadruple that of the source operand. The data width of source operand is in instruction encoding.

- mfmacc.s.<e4m3/e5m2>: fp8(e4m3/e5m2) floating-point source and fp32 result ,illegal if 'mmf8f32' of xmisa register is 0.

```
#float matrix multiplication, output quad_widen, md = md + ms1*ms2
th.mfmacc.s.e4m3 md, ms2, ms1
th.mfmacc.s.e5m2 md, ms2, ms1
```

For mfmacc.s.<e4m3/e5m2>, the legal matrix shape is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/8
- matrixB: N ≤ RLEN/32, K ≤ RLEN/8
- matrixC: M ≤ RLEN/32, N ≤ RLEN/32

Summary for max matrix size of quad-widen mfmacc instructions for typical RLEN:

		matrix A			**matrix B**			**matrix C**		
	RLEN	M	K	matrix width	N	K	matrix width	M	N	matrix width
mfmacc.s.<e5m2/e4m3>	128	4	16	512 bits	4	16	512 bits	4	4	512 bits
	256	8	32	2048 bits	8	32	2048 bits	8	8	2048 bits
	512	16	64	8192 bits	16	64	8192 bits	16	16	8192 bits

5.2.4. Integer Matrix Multiplication

The integer matrix multiplication with destination data width is quad-widen that of the source data width. The source operand data width in instruction encoding supported are int8, other data widths are reserved. Both signed/unsigned versions are provided . Thus, the source operand can be both signed/both unsigned/signed-unsigned/unsigned-signed, the result of multiplication is sign-extended before addition and accumulation. The overflow is processed depends on the msaten mode. If the msaten is 1, the overflow result should be saturated to the maximum number, and set the xmsat bit. If the msaten is 0, the overflow result should be wrap around.

- mmacc.w.b/mmaccu.w.b/mmaccus.w.b/mmaccsu.w.b: int8 quad-widen matrix multiplication, illegal if mmi8i32 of xmisa register is 0.

```
#float matrix multiplication, output quad_widen, md = md + ms1*ms2
th.mmacc.w.b   md, ms2, ms1          #signed matrix multiply
th.mmaccu.w.b  md, ms2, ms1          #unsigned matrix multiply
th.mmaccus.w.b md, ms2, ms1          #unsigned(ms1)-signed(ms2) matrix multiply
th.mmaccsu.w.b md, ms2, ms1          #signed(ms1)-unsigned(ms2) matrix multiply
```

For int8 quad-widen matrix-multiplication, the maximum matrix shape is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/8
- matrixB: N ≤ RLEN/32, K ≤ RLEN/8
- matrixC: M ≤ RLEN/32, N ≤ RLEN/32

Summary for max matrix size of quad-widen mmacc instructions for typical RLEN:

		matrix A			matrix B			matrix C		
	RLEN	M	K	matrix width	N	K	matrix width	M	N	matrix width
mmacc.w.b	128	4	16	512 bits	4	16	512 bits	4	4	512 bits
	256	8	32	2048 bits	8	32	2048 bits	8	8	2048 bits
	512	16	64	8192 bits	16	64	8192 bits	16	16	8192 bits

5.3. Load and Store Instructions

Matrix load instructions load a matrix tile from memory to matrix register. and matrix store instructions store a matrix tile from matrix register to memory.

The element data width is in instruction encoding, including 8/16/32/64 bits, other data widths are reserved. The base address is in rs1 and row stride in byte is in rs2, md/ms3 is the register index for destination of matrix load and source for matrix store.

Matrix shape (MxK) is in matrix size configure register, M given by sizeM and K given by sizeK/element size(in byte). The legal matrix shape is:

- sizeM ≤ RLEN/32
- sizeK ≤ RLEN/8, must be multiple of element size(in byte)

If sizeM < RLEN/32 or sizeK < RLEN/8, the matrix register data with row index > sizeM or column index > (sizeK/ element size(in byte)) set zero for load, and don’t write to memory for store.

Stream load/store instructions have the same function as normal matrix load/store instructions, except that the data may not be reused in the near future which can be potentially optimized by hardware implementation.

In addition, considering that matrices are stored in memory either in row-major or column-major order, load/store instructions provide both non-transposed and transposed instructions. For non-transposed instructions, the matrix tile in memory is row-major of A, BT, and C matrix. For transposed instructions, the matrix tile in memory is column major.

For matrix load/store instructions, the destination matrix register or the source matrix register can be m0-m7.

For all load/store instructions excluding the whole matrix load and store instructions, sizeM should be less than RLEN/32, sizeK should be less than RLEN/8 and should be a multiple of the element byte width. It should be noted that when sizeM is zero, all sizeK is legal, and when sizeK is zero, all sizeM is legal.

5.3.1. Load Instructions

```
# md destination, rs1 is base address, rs2 is byte stride

# matrix load instruction
th.mlde8  md, rs2, (rs1)
th.mlde16 md, rs2, (rs1)
th.mlde32 md, rs2, (rs1)
th.mlde64 md, rs2, (rs1)
```

5.3.2. Store Instructions

```
# ms3 store data, rs1 is base address, rs2 is byte stride

#matrix store instructions
th.mste8  ms3, rs2, (rs1)
th.mste16 ms3, rs2, (rs1)
th.mste32 ms3, rs2, (rs1)
th.mste64 ms3, rs2, (rs1)
```

5.3.3. Transposed Load Instructions

```
# md destination, rs1 is base address, rs2 is byte stride

#matrix transposed load instructions
th.mldte8  md, rs2, (rs1)
th.mldte16 md, rs2, (rs1)
th.mldte32 md, rs2, (rs1)
th.mldte64 md, rs2, (rs1)
```

5.3.4. Transposed Store Instructions

```
# ms3 store data, rs1 is base address, rs2 is byte stride

#matrix transposed store instructions
th.mstte8  ms3, rs2, (rs1)
th.mstte16 ms3, rs2, (rs1)
th.mstte32 ms3, rs2, (rs1)
th.mstte64 ms3, rs2, (rs1)
```

5.3.5. Stream Load Instructions

```
# md destination, rs1 is base address, rs2 is byte stride

# matrix stream load instruction
th.mslde8  md, rs2, (rs1)
th.mslde16 md, rs2, (rs1)
th.mslde32 md, rs2, (rs1)
th.mslde64 md, rs2, (rs1)
```

5.3.6. Stream Store Instructions

```
# ms3 store data, rs1 is base address, rs2 is byte stride

#matrix stream store instructions
th.msste8  ms3, rs2, (rs1)
th.msste16 ms3, rs2, (rs1)
th.msste32 ms3, rs2, (rs1)
th.msste64 ms3, rs2, (rs1)
```

5.3.7. Transposed Stream Load Instructions

```
# md destination, rs1 is base address, rs2 is byte stride

# matrix transposed stream load instruction
th.msldte8  md, rs2, (rs1)
th.msldte16 md, rs2, (rs1)
th.msldte32 md, rs2, (rs1)
th.msldte64 md, rs2, (rs1)
```

5.3.8. Transposed Stream Store Instructions

```
# ms3 store data, rs1 is base address, rs2 is byte stride

#matrix transposed stream store instructions
th.msstte8  ms3, rs2, (rs1)
th.msstte16 ms3, rs2, (rs1)
th.msstte32 ms3, rs2, (rs1)
th.msstte64 ms3, rs2, (rs1)
```

5.3.9. Whole Register Load & Store Instructions

Whole register load/store data with maximum matrix size from/to memory with sizeM = RLEN/32 and sizeK = RLEN/8. The matrix size configurations are ignored.

A whole register load/store instruction can operate 1/2/4/8 matrix registers. And md/ms3 register index should be aligned with the register number.

```
# md destination, rs1 is base address
#whole matrix load
th.mld<1/2/4/8>m<e8/e16/e32/e64> md,  (rs1)

# ms3 store data, rs1 is base address
#whole matrix store
th.mst<1/2/4/8>m<e8/e16/e32/e64> ms3, (rs1)
```

Whole matrix load and store instructions are intended to be used to save and restore matrix registers when the type or size of the current contents of the matrix register is not known, or where modifying xmsize would be costly. Examples include compiler register spills, matrix function calls where values are passed in matrix registers, interrupt handlers, and OS context switches. Software can determine the number of bytes transferred by reading the xmlenb register.

5.3.10. Prefetch Load Instructions

Prefetch load instructions are provide to improve the cache performance, like reducing latency, improving parallelism for executing unit and load/store unit. There is no matrix destination register, because the prefetch load instructions only load the matrix data from memory to cache. And the rs1 is the base address, rs2 is the row byte stride.

There exist normal prefetch load instructions and transposed prefetch load instructions.

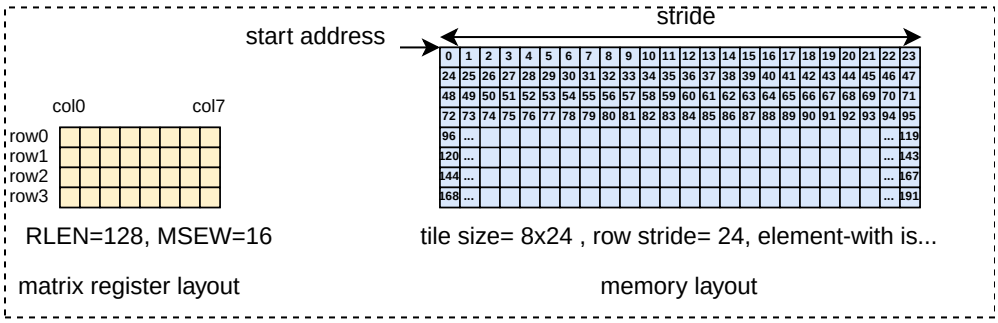
```
# rs1 base address, rs2 row byte stride

# prefetch load instructions
th.mplde8  rs2, (rs1)
th.mplde16 rs2, (rs1)
th.mplde32 rs2, (rs1)
th.mplde64 rs2, (rs1)
```

```
# rs1 base address, rs2 row byte stride

# transposed prefetch load instructions
th.mpldte8  rs2, (rs1)
th.mpldte16 rs2, (rs1)
```

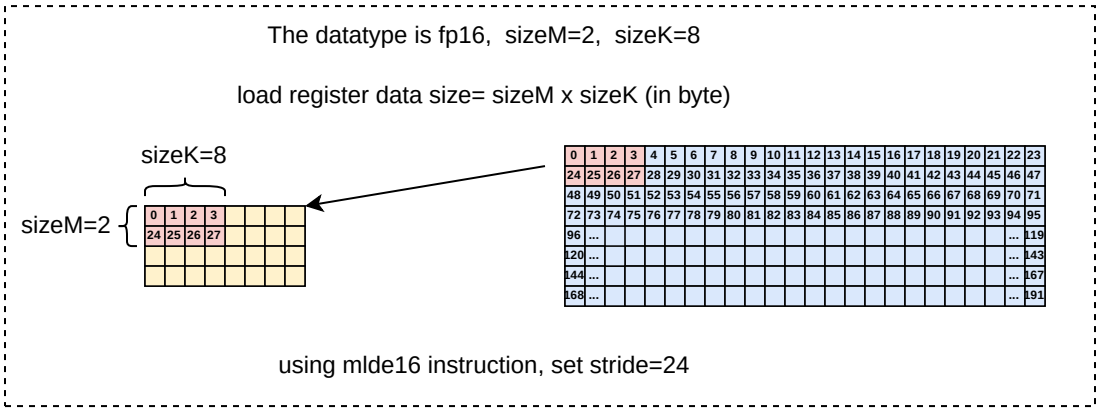
```
th.mpldte32 rs2, (rs1)
th.mpldte64 rs2, (rs1)
```

5.3.11. Examples for Load and Store Instructions

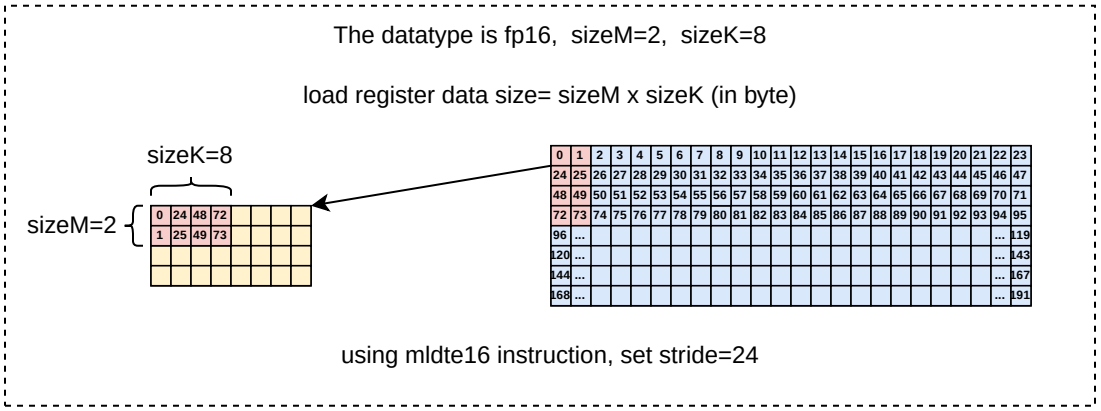
The memory layout and matrix register layout is shown below:



When sizeM is 2 and sizeK is 8, the actual matrix register size has 2 rows and 4 columns for fp16 type. The mlde16 operation is shown below:



When sizeM is 2 and sizeK is 8, the actual matrix register size has 2 rows and 4 columns for fp16 type. The mldte16 operation is shown below:



5.4. MISC Instructions

5.4.1. Mzero Instructions

Mzero instruction sets the destination register to zero. imm3 field is reused to specify the register number, encoding how many matrix registers to zero. md register index should be aligned with the register number.

imm3	register number
000	1
001	2
011	4
111	8
others	reserved

```
#zero 1 matrix register
th.mzero md
#zero 2 matrix registers
th.mzero2r md
#zero 4 matrix registers
th.mzero4r md
#zero 8 matrix registers
th.mzero8r md
```

5.4.2. Data Move Instructions

Matrix data move Instructions are provided to define the data movement between scalar registers and matrix registers. Matrix data move instructions ignore the matix size configuration.

Data Move Instructions between Matrix Registers

The mmov.mm instruction moves a whole matrix register to another matrix register.

```
#matrix-matrix mov
th.mmov.mm md, ms1

#matrix-vector mov
th.mmov.mv.i md, ms1[imm3]
```

Data Move Instructions between Integer and Matrix

Data move instructions between matrix registers and integer registers are used to move elements between matrix registers and integer registers.

The mdup<b/h/w/d>.m.x instruction moves and duplicates a scalar data to every element of the destination matrix register.

```
#matrix-scalar mov with duplicate
th.mdup<b/h/w/d>.m.x md, rs2
```

The mmov<b/h/w/d>.m.x instruction moves a scalar data to an element of the destination matrix register. The elements number within a matrix row is selected by rs1, modulo the number of such elements in a row. The row number is selected by rs1 , divided by the number of such elements in a row. The low log2(xmlenb/ element size) bits are used.

The scalar data is taken from the scalar x register specified by rs2 with XLEN data width.If data width < XLEN, the least-significant bits are copied and the upper bits are ignored. If data width > XLEN, the value is sign-extended.

```
#matrix-scalar move
th.mmov<b/h/w/d>.m.x md, rs2, rs1
```

mmov<b/h/s/d>.x.m instruction moves a scalar data from a matrix register to a gerneral purpose register specified by rd.

The scalar data is indexed by rs1. The elements number within a matrix row is selected by rs1, modulo the number of such elements in a row. The row number is selected by rs1, divided by the number of such elements in a row. The low log2(xmlenb/ element size) bits of rs1 are used.

If data width > XLEN, the least-significant XLEN bits are transferred and the upper bits are ignored. If data width < XLEN, the value is sign-extended to XLEN bits.

```
#scalar-matrix move
th.mmov<b/h/w/d>.x.m rd, ms2, rs1
```

5.4.3. Data Broadcast Instructions

The mmov.mv.i instruction is matrix register row broadcast instructions, which moves and duplicates a vector to every row of the destination matrix register. The vector data is a row of matrix register, indexed by uimm3.The log2 (RLEN/32) bits are used.

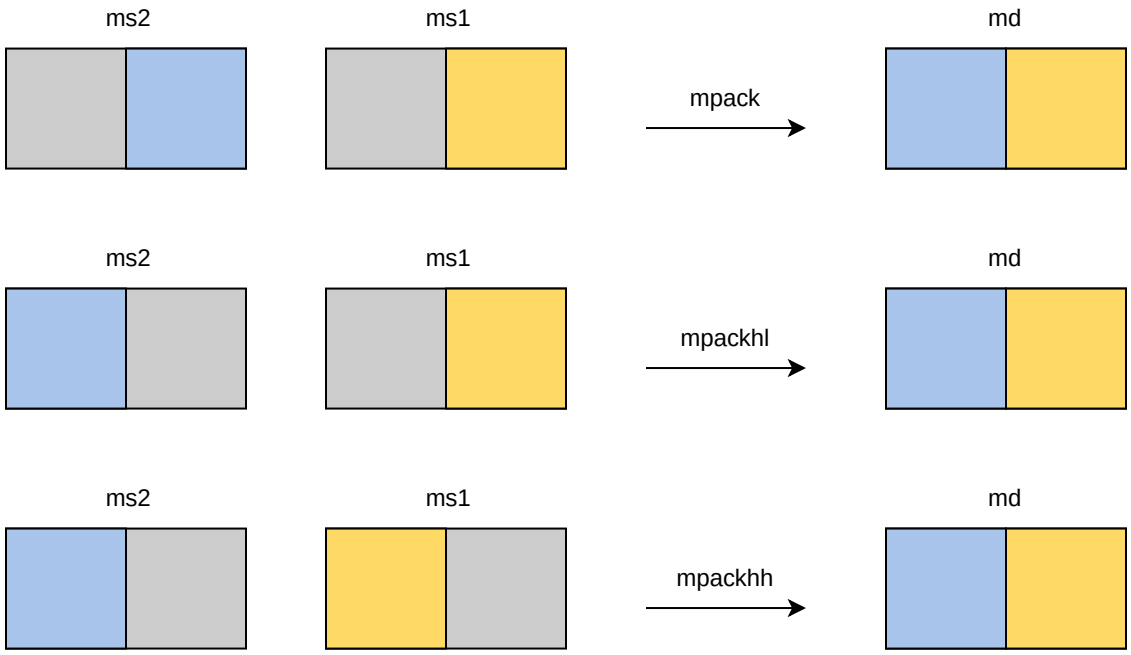
```
#matrix-vector mov
th.mmov.mv.i md, ms1[imm3]
```

The mcmov.mv.i instruction is matrix register column broadcast instructions, which duplicate the column[uimm3] of ms1 to all the columns of md. The columns are divided by MSEW so that an RLEN-bits row contains RLEN/MSEW columns. Only log2(RLEN/MSEW) bits of uimm3 are used. The matrix size configurations are ignored.

```
#matrix-matrix cast
th.mcmov<b/h/w/d>.mv.i md, ms1[uimm3]
```

5.4.4. Matrix Pack Instructions

Matrix register pack insructions merge half column of ms1 and ms2. The matrix size configurations are ignored.




```
#pack the low part of ms2 and low part of ms1 into md
th.mpack.mm md, ms2, ms1

#pack the high part of ms2 and low part of ms1 into md
th.mpackhl.mm md, ms2, ms1

#pack the high part of ms2 and high part of ms1 into md
th.mpackhh.mm md, ms2, ms1
```

5.4.5. Matrix Slide Instructions

Matrix register row slide down instructions move the row[i+imm3] of ms1 to the row[i] of md for each i from 0 to MROW-imm3-1. The remaining rows of md are set to zero. Only log2(MROW) bits of imm3 are used. The matrix size configurations are ignored.

Matrix register row slide up instructions move the row[i-imm3] of ms1 to the row[i] of md for each i from imm3 to MROW-1. The low imm3 rows of md are set to zero. Only log2(MROW) bits of imm3 are used. The matrix size configurations are ignored.

```
#matrix row slidedown, md[i,:]=ms1[i+imm3,:], when i > MROW-imm3-1, md[i,:] is zero
th.mrslidedown md, ms1, imm3

#matrix row slideup, md[i,:]=ms1[i-imm3,:], when i < imm3, md[i,:] is zero
th.mrslideup md, ms1, imm3
```

Matrix register column slidedown instructions move the column[n] of ms1 to the column[n-uimm3] of md for each n from uimm3 to RLEN/MSEW-1, where the columns are divided by MSEW so that an RLEN-bits row contains RLEN/MSEW columns. The remaining columns of md are set to zero. Only log2(RLEN/MSEW) bits of uimm3 are used. The matrix size configurations are ignored.

Matrix register column slideup instructions move the column[n-uimm3] of ms1 to the column[n] of md for each n from uimm3 to RLEN/MSEW-1, where the columns are divided by MSEW so that an RLEN-bits row contains RLEN/MSEW columns. The remaining columns of md are set to zero. Only log2(RLEN/MSEW) bits of uimm3 are used. The matrix size configurations are ignored.

```
#matrix-matrix column slide

#md_column[n-uimm3]=ms1_column[n]
th.mcslidedown.<b/h/w/d> md, ms1, uimm3

#md_column[n]=ms1_column[n-uimm3]
th.mcslideup.<b/h/w/d> md, ms1, uimm3
```

5.5. Element-Wise Instructions

5.5.1. Integer Arithmetic Element-Wise Instructions

For integer arithmetic element-wise instructions, matrix-matrix/matrix-vector instruction format are provided. 32-bit integer instructions are optionally supported.

- 32-bit width operand: legal if 'miew' and 'mew32b' of xmisa register are both set to 1

The matrix operand shape is configured by sizeM and sizeK. The legal matrix shapes for 32-bit width operand are:

- sizeM ≤ RLEN/32
- sizeK ≤ RLEN/8, sizeK must be multiples of element size in byte

The elements out of bound of `sizeM x sizeK` are set to 0.

For matrix-matrix instructions, both sources are matrix. For matrix-vector instructions, src2 is matrix and src1 is one row of matrix. Use row0 as an example, the vector operand operates on each row of matrix operand as md[i, j] = ms2[i, j] op ms1[0, j].

madd performs the addition of src1 and src2. msub performs the subtraction of src2 from src1. The overflows are ignored and the low MSEW bits of results are written to the destination md. mmul performs the multiplication of src1 and src2. The mmul versions write the low bits of the product to the destination register, while the mmulh versions write the high bits of the product to the destination register. mmax and mmin perform signed/unsigned compares, writing the max or min element to the destination respectively.

```
#matrix-matrix add
th.madd.w.mm md, ms2, ms1
#matrix-vector add
th.madd.w.mv.i md, ms2, ms1[imm3]

#matrix-matrix sub
th.msub.w.mm md, ms2, ms1
#matrix-vector sub
th.msub.w.mv.i md, ms2, ms1[imm3]

#matrix-matrix mul
th.mmul.w.mm md, ms2, ms1
#matrix-vector mul
th.mmul.w.mv.i md, ms2, ms1[imm3]

#matrix-matrix mulh
th.mmulh.w.mm md, ms2, ms1
#matrix-vector mulh
th.mmulh.w.mv.i md, ms2, ms1[imm3]

#matrix-matrix max
th.mmax.w.mm md, ms2, ms1
#matrix-vector max
th.mmax.w.mv.i md, ms2, ms1[imm3]

#matrix-matrix unsigned max
th.mumax.w.mm md, ms2, ms1
#matrix-vector unsigned max
th.mumax.w.mv.i md, ms2, ms1[imm3]

#matrix-matrix min
th.mmin.w.mm md, ms2, ms1
#matrix-vector min
th.mmin.w.mv.i md, ms2, ms1[imm3]

#matrix-matrix unsigned min
th.mumin.w.mm md, ms2, ms1
```

```
#matrix-vector unsigned min
th.mumin.w.mv.i md, ms2, ms1[imm3]
```

Matrix shift instructions including msll/msrl/msra. msll msrl and msra perform logical left logic right and arithmetic right shift, the source data is in ms2, and the shift amount is provided by a matrix/vector data specified by ms1/ms1[imm3].

```
#matrix-matrix logical left shift
th.msll.w.mm md, ms2, ms1
#matrix-vector logical left shift
th.msll.w.mv.i md, ms2, ms1[imm3]

#matrix-matrix logic right shift
th.msrl.w.mm md, ms2, ms1
#matrix-vector logic right shift
th.msrl.w.mv.i md, ms2, ms1[imm3]


#matrix-matrix arithmetic right shift
th.msra.w.mm md, ms2, ms1
#matrix-vector arithmetic right shift
th.msra.w.mv.i md, ms2, ms1[imm3]
```

5.5.2. Float-point Arithmetic Element-Wise Instructions

For float-point arithmetic element-wise instructions, matrix-matrix/matrix-vector instruction format are provided. 16-bit, 32-bit and 64-bit floating point element-wise operations are optionally supported. B

- fp16 operand: legal if `mfew` and `mew16b` are both set to 1
- fp32 operand: legal if `mfew` and `mew32b` are both set to 1
- fp64 operand: legal if `mfew` and `mew64b` are both set to 1

The matrix operand shape is configured by sizeM and sizeK. The elements out of bound of `sizeM x sizeK` are set to 0.

The legal matrix shapes for 16-bit width operand are:

- sizeM ≤ RLEN/32
- sizeK ≤ RLEN/8, and must be multiples of 2

The legal matrix shapes for 32-bit width operand are:

- sizeM ≤ RLEN/32

- sizeK ≤ RLEN/8, and must be multiples of 4

The legal matrix shapes for 64-bit width operand are:

- sizeM ≤ RLEN/32
- sizeK ≤ RLEN/8, and must be multiples of 8

For matrix-matrix instructions, both sources are matrixs. For matrix-vector instructions, src2 is matrix and src1 is one row of matrix. Use row0 as an example, the vector operand operates on each row of matrix operand as md[i, j] = ms2[i, j] operations with ms1[0, j].

All floating-point element-wise instructions follow the IEEE-754/2008 standard. All floating-point element-wise instructions that perform rounding can select the rounding mode using the frm field.

Non-widen float element-wise instructions indicate the source and destination operands data width keep the same which are encoded in the instruction.

```
#matrix-matrix fadd
th.mfadd.<h/s/d>.mm md, ms2, ms1
#matrix-vector fadd
th.mfadd.<h/s/d>.mv.i md, ms2, ms1[uimm3]

#matrix-matrix fsub
th.mfsub.<h/s/d>.mm md, ms2, ms1
#matrix-vector sub
th.mfsub.<h/s/d>.mv.i md, ms2, ms1[uimm3]

#matrix-matrix fmul
th.mfmul.<h/s/d>.mm md, ms2, ms1
#matrix-vector fmul
th.mfmul.<h/s/d>.mv.i md, ms2, ms1[uimm3]

#matrix-matrix fmax
th.mfmax.<h/s/d>.mm md, ms2, ms1
#matrix-vector fmax
th.mfmax.<h/s/d>.mv.i md, ms2, ms1[uimm3]

#matrix-matrix fmin
th.mfmin.<h/s/d>.mm md, ms2, ms1
#matrix-vector fmin
th.mfmin.<h/s/d>.mv.i md, ms2, ms1[uimm3]
```

5.5.3. Type-Convert Element-Wise Instructions

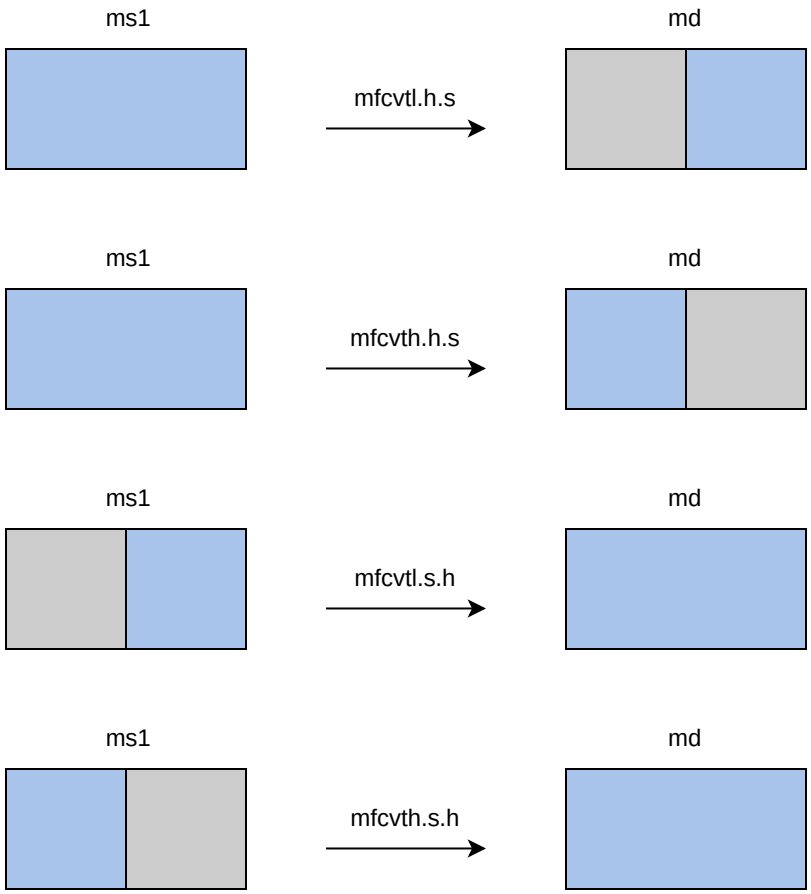
The type convert instructions includes float-point convert to float-point, float-point convert to integer, integer convert to float-point, and fix-point convert to fix-point. And the matrix size configurations are ignored.

Float-point Conversion Element-Wise Instructions

The convert instruction converts each element from ms1 to destination register md. Narrow conversion is supported by specifying the source operand width and result operand width, source elements are twice as wide as destination elements. Low or high half columns of matrix register is used by destination matrix specified by md with another half columns kept undisturbed. Widen conversion is also supported by specifying the source operand width and result operand width, and destination elements are twice as wide as source elements. Low or high half columns of matrix register are used by source matrix specified by ms1. When the conversion result is fp8, the conversion will be affected by the xmsaten csr.

For float-point conversion element-wise instructions, if the `mfew` bit is 0, all float-point conversion element-wise instructions are illegal.

- When the `mfew`, `mew8b` and `mew16b` are all set to 1, the float-point conversion element-wise instructions between fp8(e4m3/e5m2) and fp16 are legal.
- When the `mfew`, `mew16b` and `mew32b` are all set to 1, the float-point conversion element-wise instructions between fp16/bf16 and fp32 are legal.
- When the `mfew`, `mew64b` and `mew16b` are all set to 1, the float-point conversion element-wise instructions between fp32 and fp64 are legal.
- When the `mfew`, `mew8b` and `mew32b` are all set to 1, the float-point conversion element-wise instructions from fp32 to fp8(e4m3/e5m2) are legal.

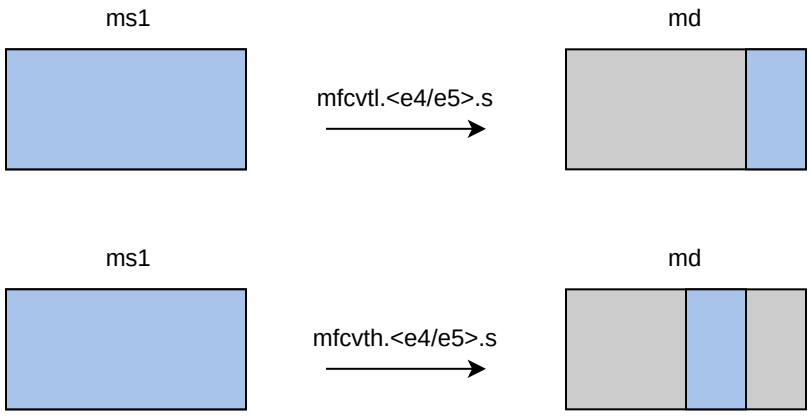



```
#matrix-matrix floating point narrow convert(fp16tofp8, fp32tofp16, fp32tobf16,
fp64tofp32)

#write back to the 2nd half of md
th.mfcvth.e4m3.h md, ms1
th.mfcvth.e5m2.h md, ms1
th.mfcvth.h.s md, ms1
th.mfcvth.bf16.s md, ms1
th.mfcvth.s.d md, ms1

#write back to the 1st half of md
th.mfcvtl.e4m3.h md, ms1
th.mfcvtl.e5m2.h md, ms1
th.mfcvtl.h.s md, ms1
th.mfcvtl.bf16.s md, ms1
th.mfcvtl.s.d md, ms1


#matrix-matrix floating point widen convert(fp8tofp16, fp16tofp32, bf16tofp32,
fp32tofp64)

#convert the 2nd half of ms1
th.mfcvth.h.e4m3 md, ms1
```

```
th.mfcvth.h.e5m2 md, ms1
th.mfcvth.s.h md, ms1
th.mfcvth.s.bf16 md, ms1
th.mfcvth.d.s md, ms1

#convert the 1st half of ms1
th.mfcvtl.h.e4m3 md, ms1
th.mfcvtl.h.e5m2 md, ms1
th.mfcvtl.s.h md, ms1
th.mfcvtl.s.bf16 md, ms1
th.mfcvtl.d.s md, ms1
```

The mfcvt<l/h>.<e5m2/e4m3>.s instructions are used to convert a single float-point source into a 4x narrower destination. The mfcvtl.<e5m2/e4m3>.s write back to the first quater of md and the mfcvth.<e5m2/e4m3>.s write back to the second quater of md, the rest parts of md are kept undisturbed.

```
#matrix-matrix floating point 4x narrow convert(fp32tofp8)

#convert the 2nd half of ms1
th.mfcvth.e5m2.s md, ms1
th.mfcvth.e4m3.s md, ms1

#convert the 1st half of ms1
th.mfcvtl.e5m2.s md, ms1
th.mfcvtl.e4m3.s md, ms1
```



Float-point Interger Conversion Element-Wise Instructions

For float-point integer conversion element-wise instructions, if the `mfic` bit is 0, all float-point integer conversion element-wise instructions are illegal.

- When the `mfic`, `mew8b` and `mew16b` are all set to 1, the float-point integer conversion element-wise instructions between int8 and fp16 are legal.

- When the mfic and mew32b are both set to 1, the float-point integer conversion element-wise instructions between int32 and fp32 are legal.

The conversion from integer to floating point supports non-widen conversion/double widen conversion. Integers support signed and unsigned integers. For double widen conversion, Low or high half columns of matrix register are used by source matrix specified by ms1.

```
#matrix-matrix unsigned integer floating point convert(uint32 to fp32)
th.mufcvt.s.w md, ms1
#matrix-matrix unsigned integer floating point widen convert (uint8 to fp16), the
low half of ms1
th.mufcvtl.h.b md, ms1
#matrix-matrix unsigned integer floating point widen convert (uint8 to fp16), the
high half of ms1
th.mufcvth.h.b md, ms1


#matrix-matrix signed integer floating point convert(sint32 to fp32)
th.msfcvt.s.w md, ms1
#matrix-matrix signed integer floating point widen convert(sint8 to fp16),the low
half
th.msfcvtl.h.b md, ms1
#matrix-matrix signed integer floating point widen convert(sint8 to fp16), the high
half
th.msfcvth.h.b md, ms1
```

The conversion from floating point to integer supports non-narrow conversion/half narrow conversion. Integers support signed and unsigned integers. For half narrow conversion, Low or high half columns of matrix register is used by destination matrix specified by md with another half columns kept undisturbed.

```
#matrix-matrix floating point unsigned integer convert(fp32 to uint32)
th.mfucvt.w.s md, ms1
#matrix-matrix floating point unsigned integer narrow convert(fp16 to uint8)
th.mfucvtl.b.h md, ms1
th.mfucvth.b.h md, ms1


#matrix-matrix floating point signed integer convert(fp32 to sint32)
th.mfscvt.w.s md, ms1
#matrix-matrix floating point signed integer narrow convert(fp16 to sint8)
th.mfscvtl.b.h md, ms1
th.mfscvth.b.h md, ms1
```

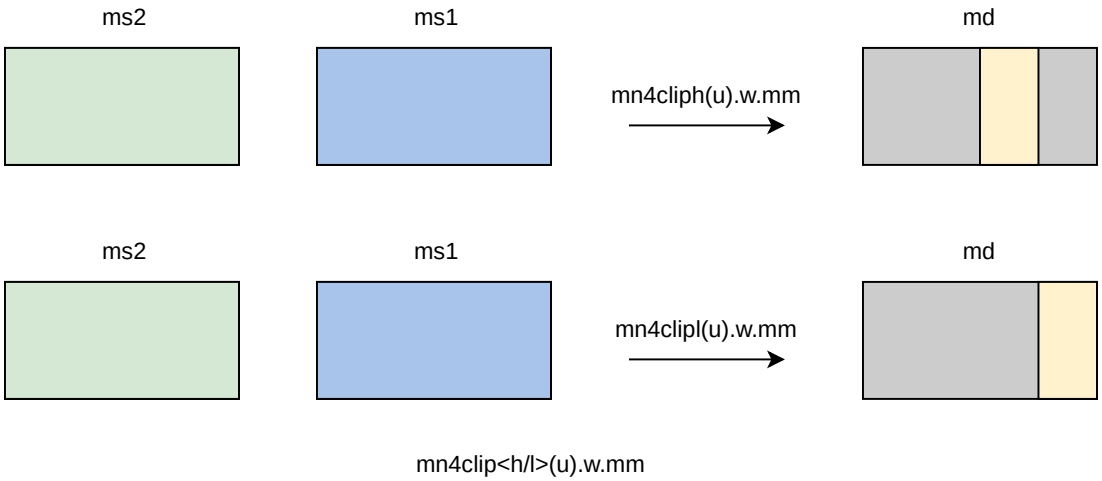
Fixed-point Conversion Element-Wise Instructions

The mn4clip instructions belong to fixed-point conversion instructions, and support rounding with rounding mode specified in the xmxrm CSR. For clip instructions, rounding occurs before saturation.

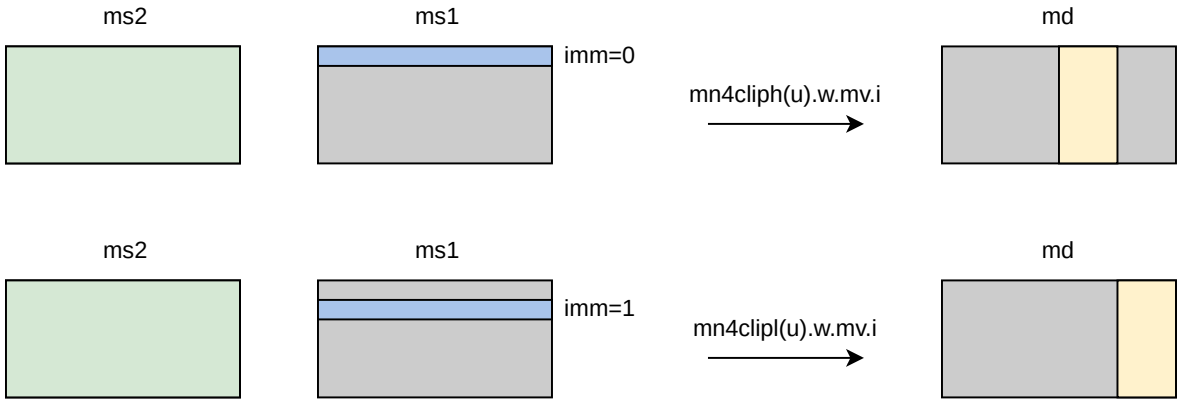
For fixed-point conversion element-wise instructions, if the `miew` bit is 0, all fixed-point conversion element-wise instructions are illegal.

- When the `miew`, `mew8b` and `mew32b` are all set to 1, the fixed-point conversion element-wise instructions from int32 to int8 are legal.

mn4cliph/mn4cliphu/mn4clipl/mn4cliplu instructions are used to pack a fixed-point value into a 4x narrower destination. Rounding, scaling and saturation are supported. The scaling shift amount comes from a matrix (specified by ms1), a vector(ms1[imm3]). The low 5-bits for 32-bit source data width are used, the higher bits are ignored. Saturation sets xmsat if the destination overflows. mn4clipl/mn4clipul write back to the first quater of md and the mn4cliph/mn4clipuh to the second quater of md, the other parts of md are kept undisturbed.



mn4clip<h/l>(u).w.mm



mn4clip<h/l>(u).w.mv.i

```
#matrix-matrix signed clip
th.mn4cliph.w.mm md, ms2, ms1
th.mn4clipl.w.mm md, ms2, ms1
#matrix-vector signed clip,uimm3
th.mn4cliph.w.mv.i md, ms2, ms1[uimm3]
th.mn4clipl.w.mv.i md, ms2, ms1[uimm3]

#matrix-matrix unsigned clip
th.mn4cliphu.w.mm md, ms2, ms1
th.mn4cliplu.w.mm md, ms2, ms1
#matrix-vector unsigned clip,uimm3
th.mn4cliphu.w.mv.i md, ms2, ms1[uimm3]
th.mn4cliplu.w.mv.i md, ms2, ms1[uimm3]
```

Implementation-defined Rule for FP8/FP4 Convert

In the OCP specification, there are some implementation-defined conversion rules for the OCP data types. The chapter describes the conversion rules for the undefined case in the OCP specification.

Input	Output	Exception
FP16/FP32 convert to E4M3, SAT mode		
out of range	max norm	OF,NX
inf	max norm	
sNaN	cNaN	NV
qNaN	cNaN	
FP16/FP32 convert to E4M3, NONSAT mode		
out of range	cnan	NV
inf	cnan	NV
sNaN	cNaN	NV
qNaN	cNaN	
FP16/FP32 convert to E5M2, SAT mode		
out of range	max norm	OF,NX
inf	max norm	
sNaN	cNaN	NV
qNaN	cNaN	
FP16/FP32 convert to E5M2, NONSAT mode		

Input	Output	Exception
out of range	inf	OF,NX
inf	inf	
sNaN	cNaN	NV
qNaN	cNaN	
E4M3 convert to E2M1, Only SAT mode		
out of range	max norm	OF,NX
NaN	positive max norm	NV
E5M2 convert to E2M1, Only SAT mode		
out of range	max norm	OF,NX
inf	max norm	
sNaN	positive max norm	NV
qNaN	positive max norm	NV

5.6. Matrix Register Overlap

Instructions support matrix source and destination registers overlap except matrix multiplication instructions.

Chapter 6. Matrix Extension

6.1. Theadmint4: INT4 Extension

6.1.1. INT4 Matrix Multiply Extension

For int4 matrix multiplication instructions, the source operand is 4-bit width and the destination is 32-bit width. Two int4 data pair are considered as an 8-bit element, the unit of sizeK is byte, so the K should be an even value, otherwise reserved. The matrix multiplication instructions mmacc.w.p/mmaccu.w.p/mmaccus.w.p/mmaccsu.w.p are illegal if 'mmi4i32' of xmisa register is 0.

The overflow is processed depends on the msaten mode. If the msaten is 1, the overflow result should be saturated to the maximum number, and set the xmsat bit. If the msaten is 0, the overflow result should be wrap around.

The legal matrix shape of int4 matrix multiplication instructions is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/4, and must be even
- matrixB: N ≤ RLEN/32, K ≤ RLEN/4, and must be even
- matrixC: M ≤ RLEN/32, N ≤ RLEN/32

```
#signed matrix multiply, ms1 and ms2 are both signed
th.mmacc.w.p md, ms2, ms1

#unsigned matrix multiply, ms1 and ms2 are both unsigned
th.mmaccu.w.p md, ms2, ms1

#unsigned-signed matrix multiply, ms1 is unsigned and ms2 is signed
th.mmaccus.w.p md, ms2, ms1

#signed-unsigned matrix multiply, ms1 is signed and ms2 is unsigned
th.mmaccsu.w.p md, ms2, ms1
```

6.1.2. INT4 Convert Extension

The int4-int8 conversion instructions are optionally supported in Theadmint4 extension. For double widen conversion, Low or high half columns of matrix register are used by source matrix specified by ms1. The matrix size configurations are ignored.

For integer conversion element-wise instructions, if the `miew` bit is 0, all integer conversion element-wise instructions are illegal.

- When the `miew`, `mew4b` and `mew8b` are all set to 1, the integer conversion element-wise instructions from int4 to int8 are legal.

```
#matrix-matrix signed integer widen convert(sint4 to sint8)
th.mscvtl.b.p md, ms1
th.mscvth.b.p md, ms1

#matrix-matrix unsigned integer widen convert(uint4 to uint8)
th.mucvtl.b.p md, ms1
th.mucvth.b.p md, ms1
```

The int4-fp8 conversion instructions are optionally supported in Theadmint4 extension. For double widen conversions, Low or high half columns of matrix register are used by source matrix specified by ms1. The matrix size configurations are ignored.

For float-point integer conversion element-wise instructions, if the `mfic` bit is 0, all float-point integer conversion element-wise instructions are illegal.

- When the `mfic`, `mew4b` and `mew8b` are all set to 1, the float-point integer conversion element-wise instructions from int4 to fp8(e4m3/e5m2) are legal.

```
#matrix-matrix sint4 convert to fp8
th.msfcvtl.e4m3.p md, ms1
th.msfcvtl.e5m2.p md, ms1
th.msfcvth.e4m3.p md, ms1
th.msfcvth.e5m2.p md, ms1


#matrix-matrix uint4 convert to fp8
th.mufcvtl.e4m3.p md, ms1
th.mufcvtl.e5m2.p md, ms1
th.mufcvth.e4m3.p md, ms1
th.mufcvth.e5m2.p md, ms1
```

6.2. Theadmfp4: FP4 Extension

For fp4 extension, fp4 matrix multiplication instructions are provided with E2M1 input data type and fp16/bf16//fp32 output data type. The definition of fp4 aligns with the OCP Microscaling Formats (MX) Specification Version 1.0.

6.2.1. FP4 Half-Precison Matrix Multiply Extension

For mfmacc.<h/bf16>.e2m1 instructions, the source is E2M1 4-bit floating-point, and the result is fp16/bf16. Two fp4 data pair are considered as an 8-bit element, the sizeK is set as 8-bit data width, so the K should be an even value, otherwise reserved. The mfmacc.h.e2m1 instruction is illegal if 'mmf4f16' of xmisa register is 0. The mfmacc.bf16.e2m1 instruction is illegal if 'mmf4bf16' of xmisa register is 0.

The legal matrix shape of mfmacc.<h/bf16>.e2m1 is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/4, and K should be an even value
- matrixB: N ≤ RLEN/16, K ≤ RLEN/4, and K should be an even value
- matrixC: M ≤ RLEN/32, N ≤ RLEN/16

As data width for matrix B is twice that of matrix A and C, two matrix register(register-pair) are used by Matrix B specified by ms2 and ms2+1. Instructions specifying an odd-numbered ms2 is reserved.

Summary for max matrix size of mfmacc.<h/bf16>.e2m1 instructions for typical RLEN:

		matrix A			**matrix B**			**matrix C**		
	RLEN	M	K	matrix width	N	K	matrix width	M	N	matrix width
mfmacc.<h/bf16>.e2m1	128	4	32	512 bits	8	32	1024 bits	4	8	512 bits
	256	8	64	2048 bits	16	64	4096 bits	8	16	2048 bits
	512	16	128	8192 bits	32	128	16384 bits	16	32	8192 bits

```
#the input is fp4(E2M1), the output is fp16, md = md + ms1*{ms2, ms2+1}
th.mfmacc.h.e2m1 md, ms2, ms1

#the input is fp4(E2M1), the output is bf16, md = md + ms1*{ms2, ms2+1}
th.mfmacc.bf16.e2m1 md, ms2, ms1
```

6.2.2. FP4 Single-Precison Matrix Multiply Extension

For mfmacc.s.e2m1 instructions, the source is E2M1 4-bit floating-point, and the result is fp32. Two fp4 data pair are considered as an 8-bit element, the sizeK is set as 8-bit data width, so the K should be an even value, otherwise reserved. The mfmacc.s.e2m1 instruction is illegal if 'mmf4f32' of xmisa register is 0.

The legal matrix shape of mfmacc.s.e2m1 is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/4, and K should be an even value
- matrixB: N ≤ RLEN/32, K ≤ RLEN/4, and K should be an even value
- matrixC: M ≤ RLEN/32, N ≤ RLEN/32

Summary for max matrix size of mfmacc.s.e2m1 instructions for typical RLEN:

		matrix A			matrix B			matrix C		
	RLEN	M	K	matrix width	N	K	matrix width	M	N	matrix width
mfmacc.s.e2m1	128	4	32	512 bits	4	32	512 bits	4	4	512 bits
	256	8	64	2048 bits	8	64	2048 bits	8	8	2048 bits
	512	16	128	8192 bits	16	128	8192 bits	16	16	8192 bits

```
#the input is fp4(E2M1), the output is fp32, md = md + ms1*ms2
th.mfmacc.s.e2m1 md, ms2, ms1
```

6.2.3. FP4 Convert Extension

For float-point conversion element-wise instructions, if the `mfew` bit is 0, all float-point conversion element-wise instructions are illegal.

- When the `mfew`, `mew4b` and `mew8b` are all set to 1, the float-point conversion element-wise instructions between fp4 and fp8(e4m3/e5m2) are legal.

The conversion to fp4 only adopts the saturating mode, and is not affected by the xmsaten csr.

For double widen conversion, mfcvt<l/h>.<e5m2/e4m3>.e2m1, Low or high half columns of matrix register are used by source matrix specified by ms1. The matrix size configurations are ignored. For narrow conversion, mfcvt<l/h>.e2m1.<e5m2/e4m3>, source elements are twice as wide as destination elements. Low or high half columns of matrix register are used by destination matrix specified by md, and another half columns kept undisturbed. The matrix size configurations are ignored.

```
#matrix-matrix floating point narrow convert(fp8tofp4)
#write back to the 2nd half of md
th.mfcvth.e2m1.e5m2 md, ms1
th.mfcvth.e2m1.e4m3 md, ms1
#write back to the 1st half of md
th.mfcvtl.e2m1.e5m2 md, ms1
th.mfcvtl.e2m1.e4m3 md, ms1

#matrix-matrix floating point widen convert(fp4tofp8)
#convert the 2nd half of ms1
th.mfcvth.e5m2.e2m1 md, ms1
th.mfcvth.e4m3.e2m1 md, ms1
#convert the 1st half of ms1
th.mfcvtl.e5m2.e2m1 md, ms1
th.mfcvtl.e4m3.e2m1 md, ms1
```

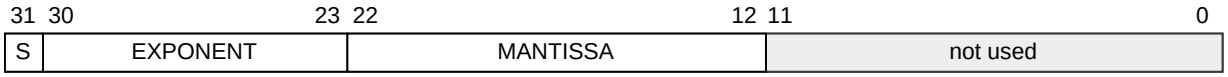
6.3. Theadmbf20: BF20 Extension

This extension provides a matrix mul-add instruction whose source operands are BF20, and accumulates into FP32, illegal if 'mmbf20f32' of xmisa register is 0.

The BF20 data type occupies 32 bits(which means the sizeK should be multiples of 4), with the following bit allocation:

- Bit 31 (most significant bit): Sign bit
- Bits 30-23: Exponent bits (8 bits)
- Bits 22-12: Mantissa bits (11 bits)
- Bits 11-0: Reserved (unused)

The layout of BF20 is as follows:



the bf20 layout

The legal matrix shape of mfmacc.s.bf20 is:

- matrixA: sizeM ≤ RLEN/32, K ≤ RLEN/32
- matrixB: sizeN ≤ RLEN/32, K ≤ RLEN/32
- matrixC: sizeM ≤ RLEN/32, N ≤ RLEN/32

```
#the input is bf20, the output is fp32, md = md + ms1*ms2
th.mfmacc.s.bf20 md, ms2, ms1
```

6.4. Theadmhp: Hybrid-Presion Extension

For hybrid precision extension, matrix multiplication operands have different precision. Three types of hybrid precison instructions are supported:

- A16W4: matrixA(Activation) is 16bit-width float-point and matrixB(Weight) is 4bit-width integer;
- A16W8: matrixA(Activation) is 16bit-width float-point and matrixB(Weight) is 8bit-width integer;
- A8W4: (1) matrixA(Activation) is 8bit-width integer and matrixB(Weight) is 4bit-width integer;
- A8W4: (2) matrixA(Activation) is 8bit-width float-point and matrixB(Weight) is 4bit-width float-point;

6.4.1. A16W4 Extension

For A16W4 extension, matrix A element width is 16bit and matrix B element width is 4bit. As data width for matrix B(4-bit) is 1/4 that of matrix A(16-bit), matrixB specified by ms2 contains 4 sizeN*(sizeK/4) matrix operands. Each instruction selects 1/2 or 1/4 columns of ms2 for multiplication by imm and accumulates to ms3 register.

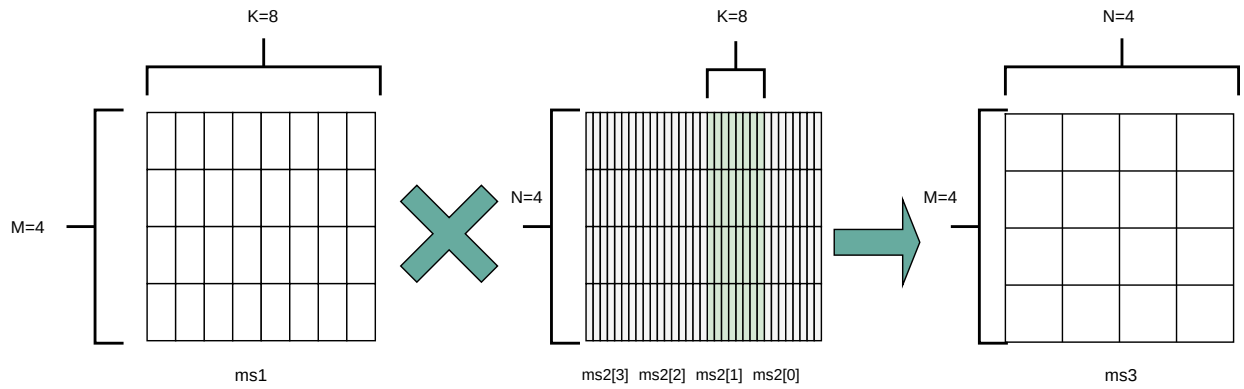
- mfmacc.h.hp: fp16*int4, output is fp16, illegal if 'mhp' of xmisa register is 0
- mfmaccu.h.hp: fp16*uint4, output is fp16, illegal if 'mhp' of xmisa register is 0
- mfmacc.s.hp: fp16*int4, output is fp32, illegal if 'mhp' of xmisa register is 0
- mfmaccu.s.hp: fp16*uint4, output is fp32, illegal if 'mhp' of xmisa register is 0

The mfmacc(u).s.hp matrix multiplication instruction select 1/4 columns from ms2. The lowest 1/4 columns of ms2 is selected by ms2[0], and other 1/4 part are selected by ms2[1], ms2p[2], and ms2[3].

The legal matrix shape of mfmacc(u).s.hp is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/16
- matrixB: N ≤ RLEN/32, K ≤ RLEN/16
- matrixC: M ≤ RLEN/32, N ≤ RLEN/32

The mfmacc(u).s.hp instruction using ms2[1] as 4-bit opernads is is shown below for RLEN=128.



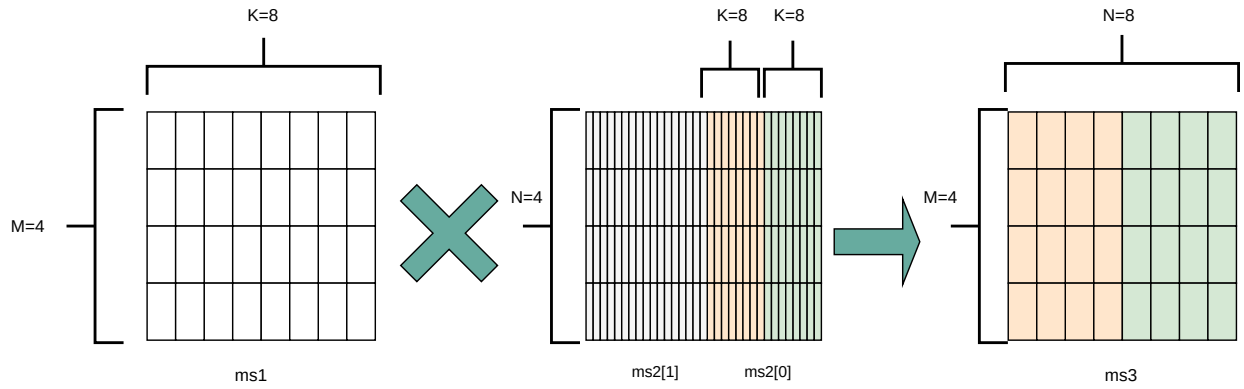
mfmacc(u).s.hp RLEN=128bit

For mfmacc(u).h.hp, 1/2 columns of ms2 are used. The low 1/4 columns from selected 1/2 ms2 columns is used to generate the low 1/2 columns of ms3, and the high 1/4 columns from selected 1/2 ms2 columns is used to generate the high 1/2 column of ms3. The lowest 1/2 columns of ms2 is ms2[0] and the highest 1/2 columns is ms2[1].

The legal matrix shape of mfmacc(u).h.hp is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/16
- matrixB: N ≤ RLEN/16, K ≤ RLEN/16
- matrixC: M ≤ RLEN/32, N ≤ RLEN/16

The instruction mfmacc(u).h.hp using ms2[0] as 4-bit operands is is shown below for RLEN=128.



mfmacc(u).h.hp RLEN=128bit

```
#fp16*int4/fp16*uint4 matrix multiplication, output fp16, md = md + ms1*ms2[imm]
th.mfmacc.h.hp   md, ms2[0/1], ms1
th.mfmaccu.h.hp  md, ms2[0/1], ms1

#fp16*int4/fp16*uint4 matrix multiplication, output fp32, md = md + ms1*ms2[imm]
th.mfmacc.s.hp  md, ms2[0/1/2/3], ms1
th.mfmaccu.s.hp md, ms2[0/1/2/3], ms1
```

6.4.2. A16W8 Extension

For A16W8 extension, matrixA element width is 16bit and matrixB element width is 8bit. As data width for matrix B(8-bit) is 1/2 that of matrix A(16-bit), matrix B specified by ms2 contains 2 sizeN*(sizeK/2) matrix operands. The instructions whose results are fp32 select 1/2 columns of ms2 for multiplication by imm and accumulates to ms3 register. The instructions whose results are fp16 use the whole ms2 register.

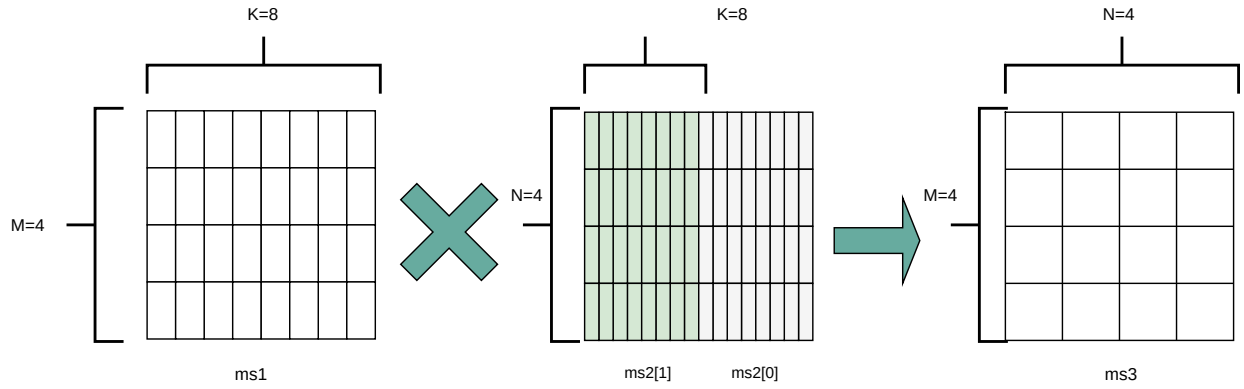
- mfmacc.h.hb: fp16*int8, output is fp16, illegal if 'mhp' of xmisa register is 0
- mfmaccu.h.hb: fp16*uint8, output is fp16, illegal if 'mhp' of xmisa register is 0
- mfmacc.s.hb: fp16*int8, output is fp32, illegal if 'mhp' of xmisa register is 0
- mfmaccu.s.hb: fp16*uint8, output is fp32, illegal if 'mhp' of xmisa register is 0

The mfmacc(u).s.hb instruction select 1/2 colunmns from ms2. The lowest 1/2 is ms2[0] and the highest 1/2 is ms2[1].

The legal matrix shape of mfmacc(u).s.hb is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/16
- matrixB: N ≤ RLEN/32, K ≤ RLEN/16
- matrixC: M ≤ RLEN/32, N ≤ RLEN/32

The mfmacc(u).s.hb instruction using ms2[1] as 8-bit opernads is is shown below for RLEN=128.



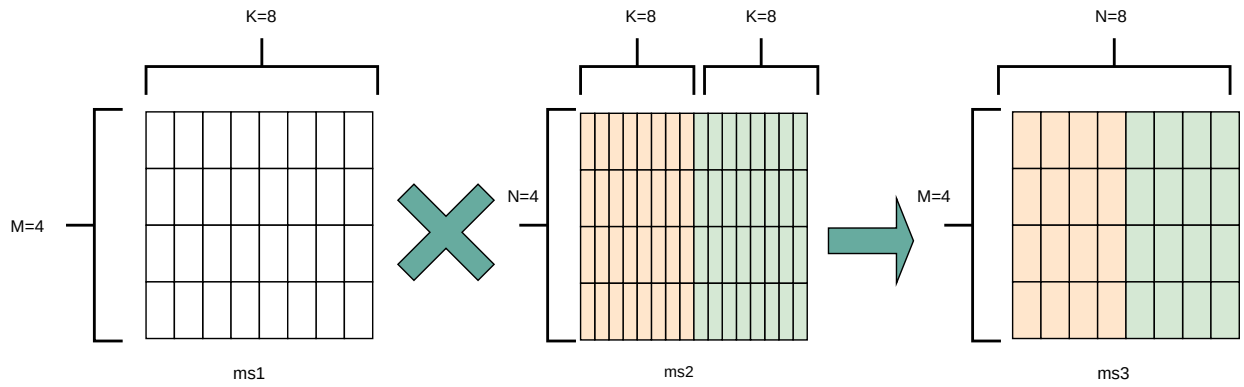
mfmacc(u).s.hb RLEN=128bit

For mfmacc(u).h.hb, all columns of ms2 are used. The low 1/2 ms2 columns is used to generate the low 1/2 columns of ms3, and the high 1/2 ms2 columns is used to generate the high 1/2 column of ms3.

The legal matrix shape of mfmacc(u).h.hb is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/16
- matrixB: N ≤ RLEN/16, K ≤ RLEN/16
- matrixC: M ≤ RLEN/32, N ≤ RLEN/16

The mfmacc(u).h.hb instruction is shown below for RLEN=128.



mfmacc(u).h.hb RLEN=128bit

```
#fp16*int8/fp16*uint8 matrix multiplication, output fp16, md = md + ms1*ms2
th.mfmacc.h.hb   md, ms2, ms1
th.mfmaccu.h.hb  md, ms2, ms1

#fp16*int8/fp16*uint8 matrix multiplication, output fp32, md = md + ms1*ms2[imm]
th.mfmacc.s.hb  md, ms2[0/1], ms1
th.mfmaccu.s.hb md, ms2[0/1], ms1
```

6.4.3. A8W4 Extension

For A8W4 extension, matrixA element width is 8bit and matrixB element width is 4bit. As data width for matrix B(4-bit) is 1/2 that of matrix A(8-bit), matrixB specified by ms2 contains 2 sizeN*(sizeK/2) matrix operands. The instructions whose results are 32-bit select 1/2 columns of ms2 for multiplication by imm and accumulates to ms3 register. The instructions whose results are 16-bit use the whole ms2 register.

The operands of A8W4 extension can be integer or float-point. For integer operand, the accumulations is only int32. For float-point operand, the accumulations can be fp16 or fp32.

- mmacc.w.bp: int8*int4, output is int32, illegal if 'mhp' of xmisa register is 0
- mmaccu.w.bp: uint8*uint4, output is int32, illegal if 'mhp' of xmisa register is 0
- mmaccus.w.bp: uint8*int4, output is int32, illegal if 'mhp' of xmisa register is 0
- mmaccsu.w.bp: int8*uint4, output is int32, illegal if 'mhp' of xmisa register is 0
- mfmacc.h.e4m3e2m1: e4m3*e2m1, output is fp16, illegal if 'mhp' of xmisa register is 0
- mfmacc.h.e5m2e2m1: e5m2*e2m1, output is fp16, illegal if 'mhp' of xmisa register is 0
- mfmacc.s.e4m3e2m1: e4m3*e2m1, output is fp32, illegal if 'mhp' of xmisa register is 0
- mfmacc.s.e5m2e2m1: e5m2*e2m1, output is fp32, illegal if 'mhp' of xmisa register is 0

For the mmacc(u/s).s.hp instructions, the overflow is processed depends on the msaten mode. If the

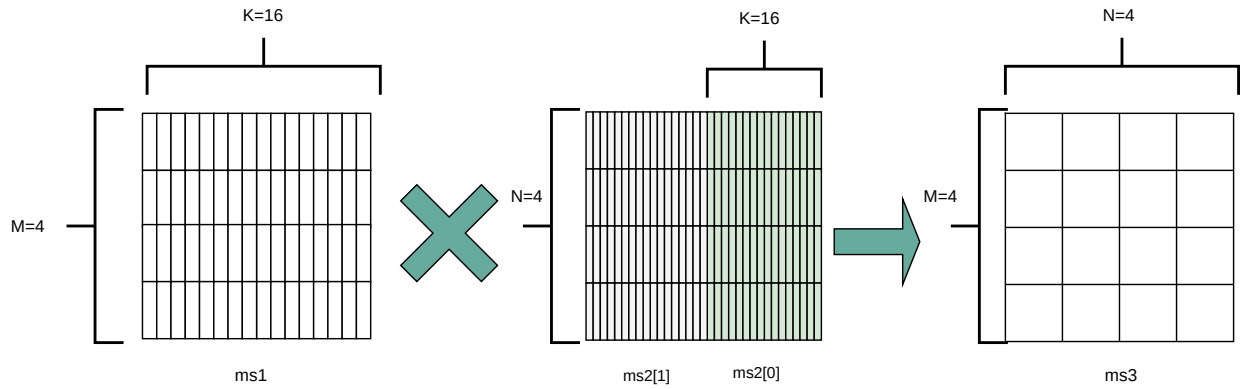
msaten is 1, the overflow result should be saturated to the maximum number, and set the xmsat bit. If the msaten is 0, the overflow result should be wrap around.

The legal matrix shape of mmacc(u/s).s.hp and mfmacc.s.(e4m3/e5m2)e2m1 is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/8
- matrixB: N ≤ RLEN/32, K ≤ RLEN/8
- matrixC: M ≤ RLEN/32, N ≤ RLEN/32

For mmacc(u/s).s.hp and mfmacc.s.(e4m3/e5m2)e2m1, the lowest 1/2 of ms2 is ms2[0] and the highest 1/2 is ms2[1].

The mmacc(u/s)(u/s).s.bp instruction using ms2[0] as 4-bit opernads is is shown below for RLEN=128.



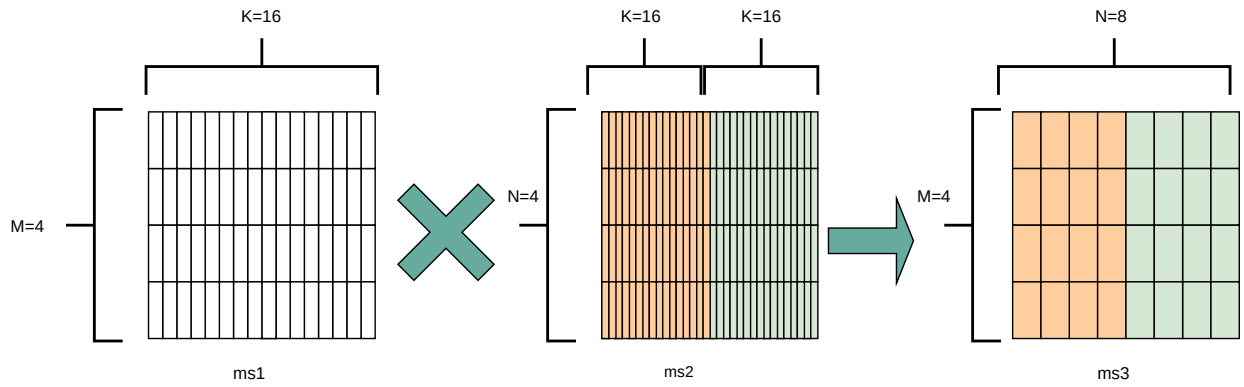
macc(u/s)(u/s).s.hp RLEN=128bit

For mfmacc.h.(e4m3/e5m2)e2m1, all columns of ms2 are used. The low 1/2 ms2 columns is used to generate the low 1/2 columns of ms3, and the high 1/2 ms2 columns is used to generate the high 1/2 column of ms3. The operation of mfmacc.h.(e4m3/e5m2)e2m1 is the same with mfmacc(u).h.hb.

The legal matrix shape of mfmacc.h.(e4m3/e5m2)e2m1 is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/8
- matrixB: N ≤ RLEN/16, K ≤ RLEN/8
- matrixC: M ≤ RLEN/32, N ≤ RLEN/16

The mfmacc.h.(e4m3/e5m2)e2m1 instruction is shown below for RLEN=128.



mfmacc.h.(e4m3/e5m2)e2m1 RLEN=128bit

```
#int8(uint8)*int4(uint4) matrix multiplication, output int32, md = md +
ms1*ms2[imm]
th.mmacc.w.bp   md, ms2[0/1], ms1   #matrix-matrix
th.mmaccu.w.bp  md, ms2[0/1], ms1   #matrix-matrix
th.mmaccus.w.bp md, ms2[0/1], ms1   #matrix-matrix
th.mmaccsu.w.bp md, ms2[0/1], ms1   #matrix-matrix

#fp8*fp4 matrix multiplication, output fp32, md = md + ms1*ms2[imm]
th.mfmacc.s.e4m3e2m1  md, ms2[0/1], ms1   #matrix-matrix
th.mfmacc.s.e5m2e2m1  md, ms2[0/1], ms1   #matrix-matrix

#fp8*fp4 matrix multiplication, output fp16, md = md + ms1*ms2
th.mfmacc.h.e4m3e2m1  md, ms2, ms1  #matrix-matrix
th.mfmacc.h.e5m2e2m1  md, ms2, ms1  #matrix-matrix
```

6.5. Theadmmx*: Microscaling(MX) Format Extension

The Theadmmx* extension supports the microscaling data format, including MXFP8, MXFP4, and MXFP8-MXFP4 hybrid-precision extension. The definition of MXFP8 and MXFP4, and the dot-product operation of microscaling data are described in “OCP Microscaling Formats (MX) Specification Version 1.0”.

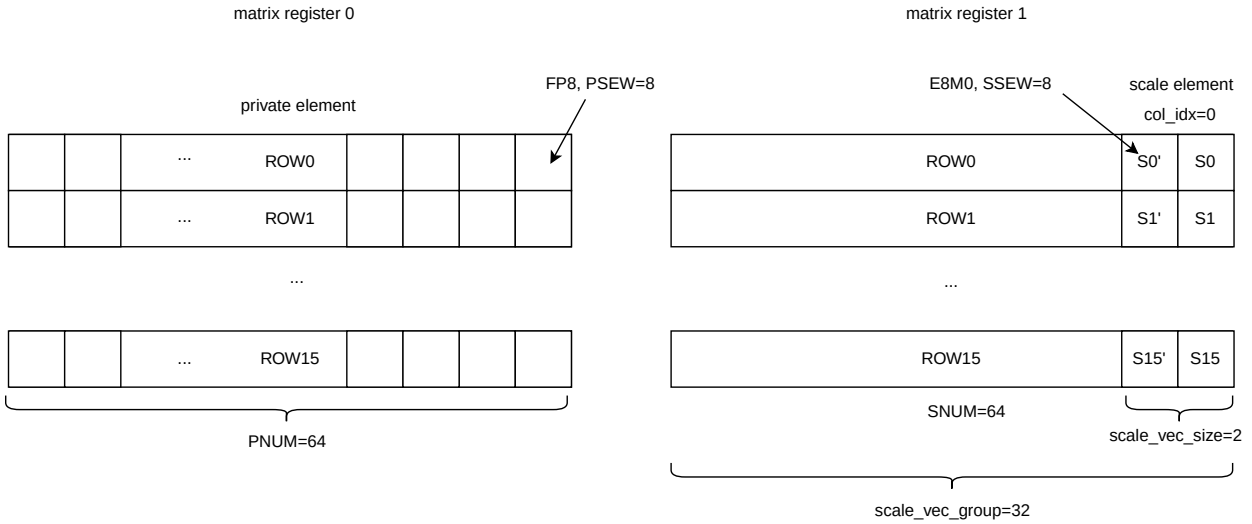
6.5.1. Microscaling Program Model

The private element of MX data are held in a matrix register, and the scale element are held in another matrix register. The scale elements for private elements in a row are stored in the same row of the scale matrix register. The following parameter are defined to describe the microscaling data format in the matrix extension:

1. PSEW: Private Element SEW, means the private element size in bits in matrix register. PSEW is the bigger one in MX hyrid-precision matrix multiplication.

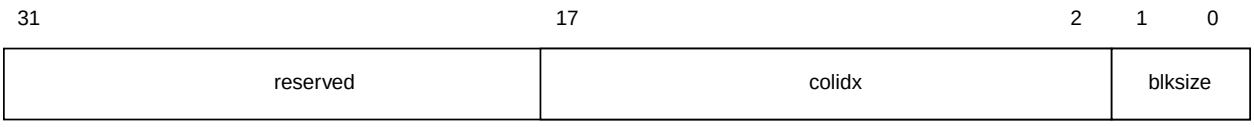
2. SSEW: Scale Element SEW, means the scale element size in bits in matrix register. The SSEW is 8 in this matrix extension, because the scale elements are E8M0 or UE4M3.
3. PNUM: Private Element NUM in a matrix row, equals to RLEN/PSEW.
4. SNUM: Scale Element NUM in a matrix row, equals to RLEN/SSEW.
5. scale_vec_size: the scale element number needed for PNUM private elements in a matrix row, equals to PNUM/blocksize.
6. scale_vec_group: the group numbers of scale_vec_size scale elements can be held in a matrix row, equals to SNUM/scale_vec_size.

The following picture shows the MXFP8 data format in two matrix registers. RLEN is 512bit, and the blocksize is 32. The private elements are stored in matrix register 0, like scaler data format. The PNUM is RLEN/PSEW=512/8=64. Each row of private element has two corresponding scale element(scale_vec_size=PNUM/blocksize=2). The scale elements are stored in matrix register 1. The SNUM is RLEN/SSEW=512/8=64. The two column scale elements are indexed by column index 0. A matrix register can hold 32 groups of the scale elements (scale_vec_group=PNUM/scale_vec_size=32).



RLEN=512bit, MXFP8, blocksize=32, col_idx=0

The blocksize of private element and the column index of scale element are configured in xmx(a/b)desc. The 'blksize' field represent four blocksize number, 16, 32, 64, 128. The 'colidx' field indicate the column index of scale elements. The column index can only be multiple of scale_vec_size.



mx(a/b)desc

For MX matrix multiplication instructions, the Matrix A is microscaling data, the Matrix B is

microscaling data, and the Matrix C is FP32 data. The instruction mnemonic of MX matrix multiplication instructions is 'mfmacc.s.mx* md, ms2, ms2.s, ms1, ms1.s'. The 'ms1' indicates the matrix register index of Matrix A's private element. The 'ms1.s' indicates the matrix register index of Matrix A's scale element. The 'ms2' indicates the matrix register index of Matrix B's private element. The 'ms2.s' indicates the matrix register index of Matrix B's scale element.

Table 5. different information of MX matrix multiplication instructions

mnemonic	meaning
ms1	matrix register index of Matrix A's private element
ms1.s	matrix register index of Matrix A's scale element
xmxadesc.blksize	blocksize of Matrix A's private element
xmxadesc.colidx	column index of Matrix A's scale element
ms2	matrix register index of Matrix B's private element
ms2.s	matrix register index of Matrix B's scale element
xmxbdesc.blksize	blocksize of Matrix B's private element
xmxbdesc.colidx	column index of Matrix B's scale element
md	matrix register index of Matrix C

The function description of MX matrix multiplication is as follows, when Matirx A's blocksize equals to Matrix B's blocksize:

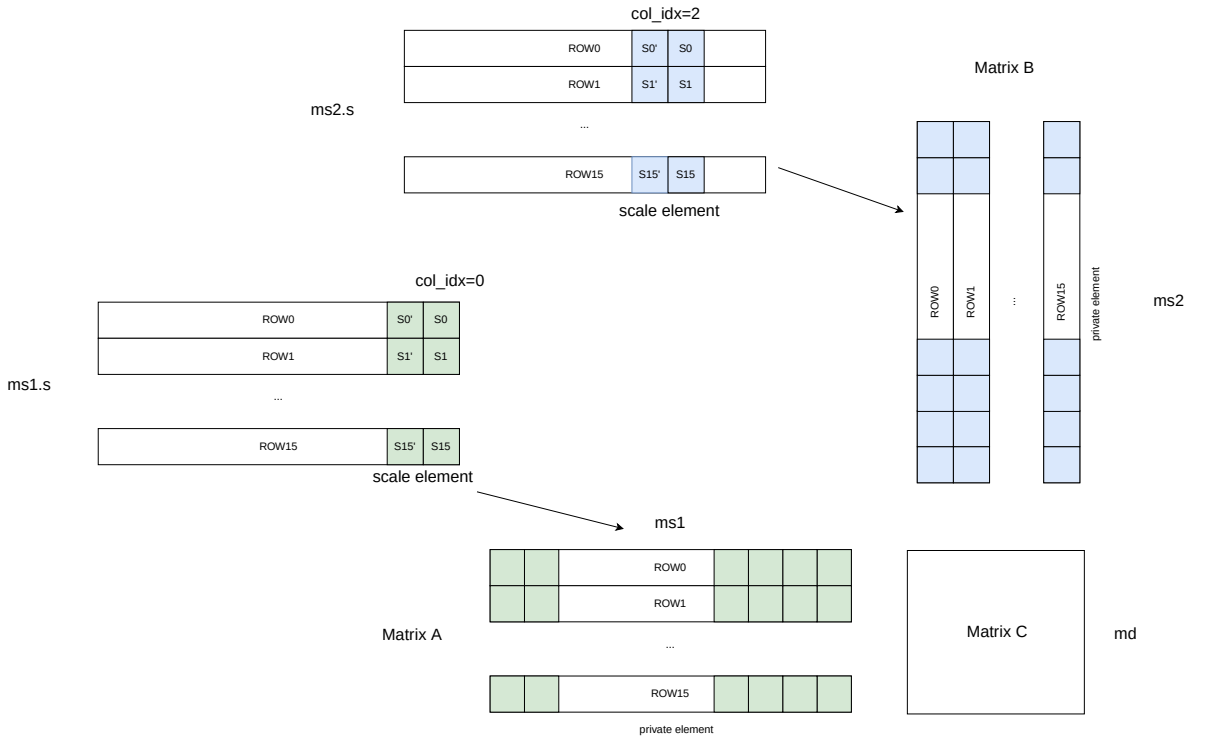
```
#A[i,k]  means the i row and k column element of ms1
#B[j,k]  means the j row and k columns element of ms2
#C[i,j]  means the i row and j column element of md
#A.s[x,y] means the x row and y column element of ms1.s
#B.s[x,y] means the x row and y column element of ms2.s
#A.colidx means the column index in xmxadesc.colidx
#A.blocksize means the actual blocksize represented by xmxadesc.blksize
#B.colidx means the column index in xmxbdesc.colidx
#B.blocksize means the actual blocksize represented by xmxbdesc.blksize
#C_p[:,i,j] is the private element dot-product result without multiplying scale
elements
#in this example, blocksize = A.blocksize = B.blocksize, sizeK is multiples of
blocksize

for(int i=0; i<sizeM; i++) {
  for(int j=0; j<sizeN; j++) {
      for(int k1=0; k1<(sizeK x 8 / PSEW)/blocksize; k1=k1+1){
         for(int k0=0; k0<blocksize; k0++){
           C_p[k1,i,j] += A[i, k1*blocksize+k0] * B[j, k1*blocksize+k0]  ;}
         C[i,j] +=  C_p[k1,i,j] * A.s[i, A.colidx+k1] * B.s[j, B.colidx+k1];
```

```
}}}
```

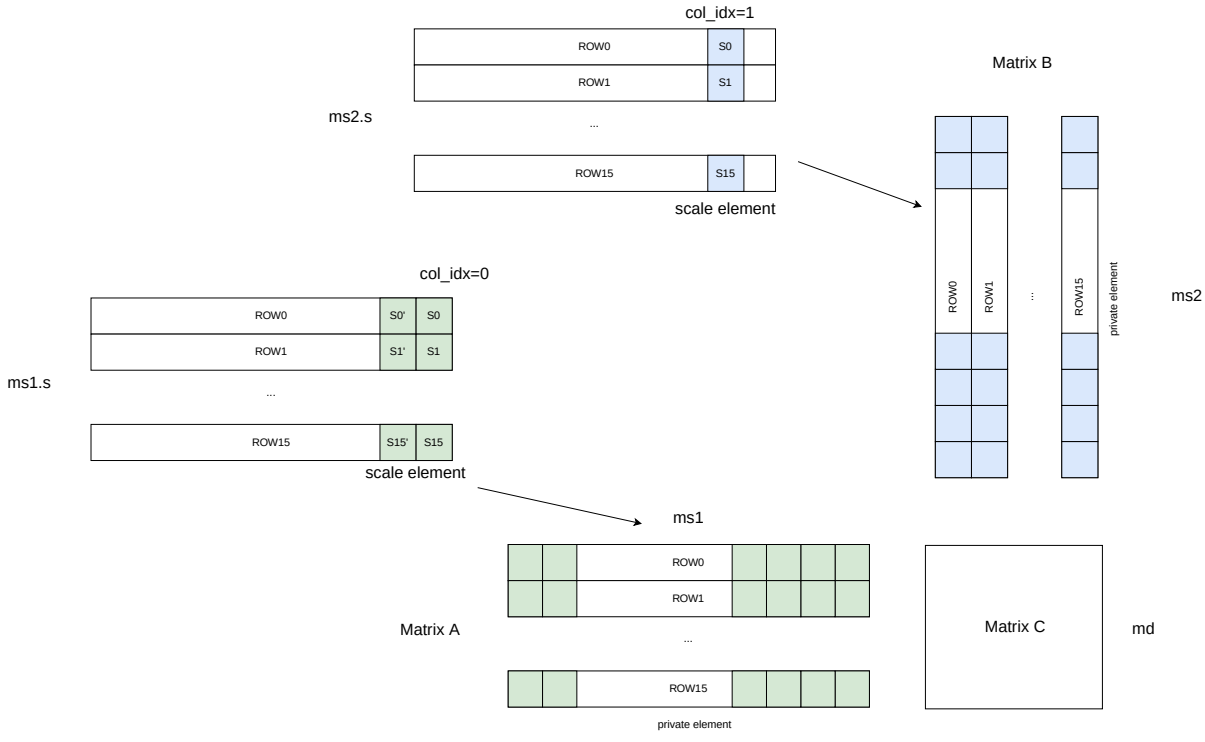
The function description of MX matrix multiplication is as follows, when Matirx A's blocksize is bigger than Matrix B's blocksize:

```
#A[i,k]  means the i row and k column element of ms1
#B[j,k]  means the j row and k columns element of ms2
#C[i,j]  means the i row and j column element of md
#A.s[x,y] means the x row and y column element of ms1.s
#B.s[x,y] means the x row and y column element of ms2.s
#A.colidx means the column index in xmxadesc.colidx
#A.blocksize means the actual blocksize represented by xmxadesc.blksize
#B.colidx means the column index in xmxbdesc.colidx
#B.blocksize means the actual blocksize represented by xmxbdesc.blksize
#C_p[:,i,j] is the private element dot-product result without multiplying scale
elements
#in this example, sizeK is multiples of A.blocksize

for(int i=0; i<sizeM; i++) {
  for(int j=0; j<sizeN; j++) {
      for(int k2=0; k2<(sizeK x 8 / PSEW)/A.blocksize; k2=k2+1){
         for(int k1=0; k1<A.blocksize/B.blocksize; k1=k1+1){
            for(int k0=0; k0<B.blocksize; k0++){
              C_p[k2*(A.blocksize/B.blocksize) + k1,i,j] +=
                           A[i, k2*A.blocksize+k1*B.blocksize+k0] *
                           B[j, k2*A.blocksize+k1*B.blocksize+k0] ;}
            C[i,j] += C_p[k2*(A.blocksize/B.blocksize) + k1,i,j] *
                      A.s[i, A.colidx+ k2] *
                      B.s[j, B.colidx+ k2*(A.blocksize/B.blocksize) + k1] ;
}}}}
```

The following picture shows the operations of MX matrix multiplication.



RLEN=512bit, MXFP8, A.blocksize=32, B.blocksize=32, A.colidx=0, B.colidx=2



RLEN=512bit, MXFP8, A.blocksize=32, B.blocksize=64, A.colidx=0, B.colidx=1

6.5.2. Microscaling Configuration Instructions

Microscaling configuration instructions configure a field or the whole microscaling descriptor csr register. The field retains the value if not changed by a configuration instruction. One configuration instruction can configure two microscaling descriptor csr, xmxadesc and xmxbdesc, by using the mnemonics 'mxa' and 'mxb'.

Table 6. the affected csr by using 'mxa' and 'mxb'

mnemonic	affected csr
no mxa, no mxb	xmxadesc, xmxbdesc
only mxa	xmxadesc
only mxb	xmxbdesc
mxa and mxb	xmxadesc, xmxbdesc

```
//imm type
th.mxcfgi.blksize uimm2, mxa, mxb  //xmx(a/b)desc[1:0] = uimm2
th.mxcfgi.colidx  uimm7, mxa, mxb  //xmx(a/b)desc[8:2] = uimm7

//register type
th.mxcfg         rs1, mxa, mxb   //xmx(a/b)desc[17:0] = rs1[17:0]
th.mxcfg.blksize rs1, mxa, mxb   //xmx(a/b)desc[1:0] = rs1[1:0]
th.mxcfg.colidx  rs1, mxa, mxb   //xmx(a/b)desc[17:2] = rs1[15:0]


//'blksize' assembler syntax
xmx(a/b)desc[1:0]  blocksize   assembler syntax
00                 16          blksize16
01                 32          blksize32
10                 64          blksize64
11                 128         blksize128
```

The following is an example of mxcfgi.blksize:

```
th.mxcfgi.blksize uimm2,     mxa, mxb  //xmxadesc[1:0] = uimm2, xmxbdesc[1:0] =
uimm2
th.mxcfgi.blksize uimm2,     mxa       //xmxadesc[1:0] = uimm2
th.mxcfgi.blksize blksize32, mxb       //xmxbdesc[1:0] = 00
th.mxcfgi.blksize blksize128           //xmxadesc[1:0] = 10, xmxbdesc[1:0] = 10
```

The mxcfg(i) instructions will not raise illegal exception. The execution of microscaling matrix multiplication instructions will check the 'colidx' and 'blksize' is legal or not. This will be described in detail in the microscaling matrix

multiplication instructions.

6.5.3. MXFP8 Matrix Multiplication Extension

For MXFP8 matrix multiplication instruction, the source operand is MXFP8 and the destination is FP32. The matrix multiplication instructions mfmacc.s.mxe4m3/mfmacc.s.mxe5m2 are illegal if 'mmmxf8' of xmisa register is 0.

The legal matrix shape of MXFP8 matrix multiplication instructions is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/8
- matrixB: N ≤ RLEN/32, K ≤ RLEN/8
- matrixC: M ≤ RLEN/32, N ≤ RLEN/32

```
#MXFP8 matrix multiplication instructions, md = {ms1, ms1.s}*{ms2, ms2.s}
th.mfmacc.s.mxe4m3 md, ms2, ms2.s ms1, ms1.s
th.mfmacc.s.mxe5m2 md, ms2, ms2.s ms1, ms1.s
```

The private element of MXFP8 is E4M3 or E5M2, and the scale element of MXFP8 is E8M0. Hence, the PSEW is 8, and the SSEW is 8. The blocksize of private elements should not bigger than PNUM. The column index can only be multiple of scale_vec_size.

Summary of actual blocksize, legal 'blksize' and 'colidx' are shown below:

RLEN	**blksize == 00**			**blksize == 01**			**blksize == 10**			**blksize == 11**		
	legal	**blocksize**	**legalcolidx**	**legal**	**blocksize**	**legalcolidx**	**legal**	**blocksize**	**legalcolidx**	**legal**	**blocksize**	**legalcolidx**
128	yes	16	%1=0	no	-	-	no	-	-	no	-	-
256	yes	16	%2=0	yes	32	%1=0	no	-	-	no	-	-
512	yes	16	%4=0	yes	32	%2=0	yes	64	%1=0	no	-	-
1024	yes	16	%8=0	yes	32	%4=0	yes	64	%2=0	yes	128	%1=0
2048	yes	16	%16=0	yes	32	%8=0	yes	64	%4=0	yes	128	%2=0

RLEN	blksize == 00			blksize == 01			blksize == 10			blksize == 11		
4096	yes	16	%32=0	yes	32	%16=0	yes	64	%8=0	yes	128	%3=0

6.5.4. MXFP4 Matrix Multiplication Extension

For MXFP4 matrix multiplication instruction, the source operand is MXFP4 and the destination is FP32. Two fp4 data pair are considered as an 8-bit element, the sizeK is set as fp8 data width, so the K should be an even value, otherwise reserved. The matrix multiplication instructions mfmacc.s.mxe2m1 is illegal if 'mmmxf4' of xmisa register is 0.

The legal matrix shape of MXFP4 matrix multiplication instructions is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/4, and must be even
- matrixB: N ≤ RLEN/32, K ≤ RLEN/4, and must be even
- matrixC: M ≤ RLEN/32, N ≤ RLEN/32

```
#MXFP4 matrix multiplication instructions, md = {ms1, ms1.s}*{ms2, ms2.s}
th.mfmacc.s.mxe2m1 md, ms2, ms2.s ms1, ms1.s
th.mfmacc.s.mxe2m1.ue4m3 md, ms2, ms2.s ms1, ms1.s
```

For mfmacc.s.mxe2m1, the private element is E2M1, and the scale element is E8M0. For mfmacc.s.mxe2m1.ue4m3, the private element is E2M1, and the scale element is UE4M3. For the MXFP4 mfmacc instructions, the PSEW is 4, and the SSEW is 8. The blocksize of private elements should not bigger than PNUM. The column index can only be multiple of scale_vec_size.

Each UE4M3 element occupies 8bit in matrix register and memory, and the meaning of UE4M3’s low 7bit is the same with E4M3, which includes 4 bits for exponent and 3bits for mantissa. The MSB bit of UE4M3 is ignored.

Summary of actual blocksize, legal 'blksize' and 'colidx' are shown below:

RLEN	blksize == 00			blksize == 01			blksize == 10			blksize == 11		
	legal	blocksize	legalcolidx	legal	blocksize	legalcolidx	legal	blocksize	legalcolidx	legal	blocksize	legalcolidx
128	yes	16	%2=0	yes	32	%1=0	no	-	-	no	-	-

RLEN	blksize == 00			blksize == 01			blksize == 10			blksize == 11		
256	yes	16	%4=0	yes	32	%2=0	yes	64	%1=0	no	-	-
512	yes	16	%8=0	yes	32	%4=0	yes	64	%2=0	yes	128	%1=0
1024	yes	16	%16=0	yes	32	%8=0	yes	64	%4=0	yes	128	%2=0
2048	yes	16	%32=0	yes	32	%16=0	yes	64	%8=0	yes	128	%4=0
4096	yes	16	%64=0	yes	32	%32=0	yes	64	%16=0	yes	128	%8=0

6.5.5. MXFP8-MXFP4 Hybrid-precison Matrix Multiplication Extension

For MXFP8-MXFP4 hybrid-precision matrix multiplication instruction, the source operand of Matrix A is MXFP8, the source operand of Matrix B is MXFP4, and the destination is FP32. The matrix multiplication instructions mfmacc.s.mxe4m3mxe2m1/mfmacc.s.mxe5m2mxe2m1 are illegal if 'mmmxf8mxf4' of xmisa register is 0.

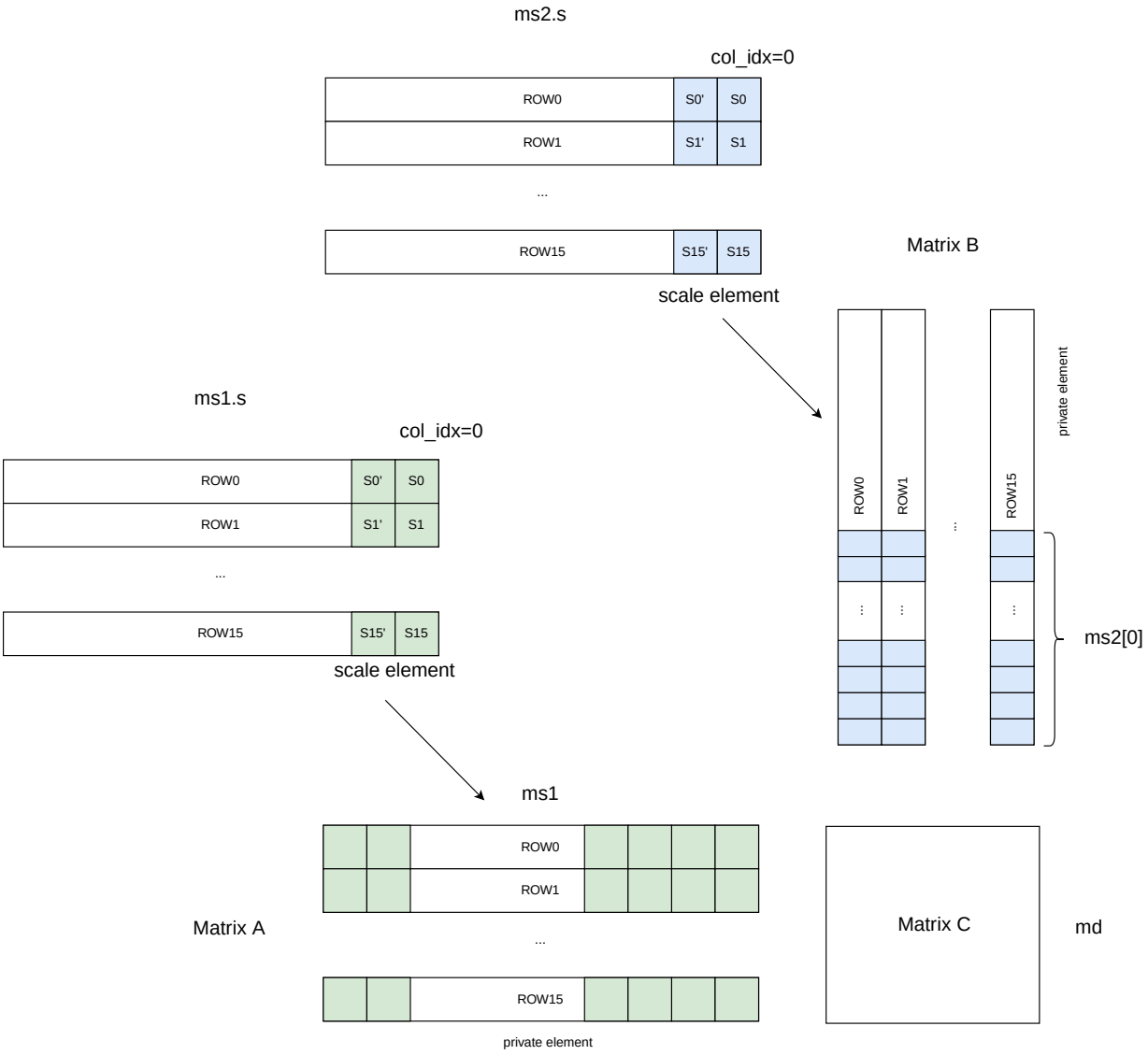
As the private element data width for Matrix B(4-bit) is 1/2 that of Matrix A(8-bit), the private element matrix register of Matrix B specified by ms2 contains 2 sizeN*(sizeK/2) matrix operands. Each instruction selects 1/2 columns of ms2 for multiplication by imm.

The legal matrix shape of MXFP8-MXFP4 hybrid-precision matrix multiplication instructions is:

- matrixA: M ≤ RLEN/32, K ≤ RLEN/8
- matrixB: N ≤ RLEN/32, K ≤ RLEN/8
- matrixC: M ≤ RLEN/32, N ≤ RLEN/32

```
#MXFP8-MXFP4 hybrid-precision matrix multiplication instruction
#md = {ms1, ms1.s}*{ms2[0/1], ms2.s}
th.mfmacc.s.mxe4m3mxe2m1 md, ms2[0/1], ms2.s ms1, ms1.s
th.mfmacc.s.mxe5m2mxe2m1 md, ms2[0/1], ms2.s ms1, ms1.s
```

The following picture shows the MXFP8-MXFP4 hybrid-precision matrix multiplication instruction, using ms2[0] as 4-bit Matrix B private element.



RLEN=512bit, MXFP8*MXFP4, A.blocksize=32, B.blocksize=32, A.colidx=0, B.colidx=0

The source operands include MXFP8 and MXFP4. The MXFP8 is selected to generate the legal blocksize because of the bigger data width. The private element of MXFP8 is E4M3 or E5M2, and the scale element of MXFP8 is E8M0. Hence, the PSEW is 8, and the SSEW is 8. The blocksize of private elements should not bigger than PNUM. The column index can only be multiple of scale_vec_size.

Summary of actual blocksize, legal 'blksize' and 'colidx' are shown below:

RLEN	blksize == 00			blksize == 01			blksize == 10			blksize == 11		
	legal	blocksize	legalcolidx	legal	blocksize	legalcolidx	legal	blocksize	legalcolidx	legal	blocksize	legalcolidx
128	yes	16	%1=0	no	-	-	no	-	-	no	-	-
256	yes	16	%2=0	yes	32	%1=0	no	-	-	no	-	-
512	yes	16	%4=0	yes	32	%2=0	yes	64	%1=0	no	-	-
1024	yes	16	%8=0	yes	32	%4=0	yes	64	%2=0	yes	128	%1=0
2048	yes	16	%16=0	yes	32	%8=0	yes	64	%4=0	yes	128	%2=0
4096	yes	16	%32=0	yes	32	%16=0	yes	64	%8=0	yes	128	%3=0

6.5.6. MX Matrix Multiplication Example

Example code for Matrix A and Matrix B with same blocksize:

```
//RLEN=512bit
//Source Operand: MXFP8
//Blocksize: 32
//A: M*K = 16*K
//B: N*K = 32*K
//C: M*N = 16*32

block_size      = 32
scale_vec_size  = 2   // PNUM/block_size = (RLEN/PSEW)/block_size = (512/8)/32 = 2
scale_vec_group = 32  // SNUM/scale_vec_size = (RLEN/SSEW)/scale_vec_size =
(512/8)/2 = 32

mcfgmi (RLEN/32)
mcfgni (RLEN/32)
mcfgki (RLEN/8)

//config the xmxadesc/xmxbdesc blksize field to 00, blocksize is 32
mxcfgi.blksize blksize32, mxa, mxb
```

```
//initialize xmxadesc.colidx and xmxbdesc.colidx
x15 = 0
mxcfg.colidx x15, mxa, mxb

x1 = Cbaseaddr
mlde32 m6, x2, (x1) //load C0
x1 = x1 + Cstride
mlde32 m7, x2, (x1) //load C1


for(k1=0; k1<K; k1+=blocksize*scale_vec_size*scale_vec_group)
{
  mlde8 m1, x4, (x3)      //load A0.scale for Matrix A0
  x3 = x3 + Asstride
  mlde8 m3, x6, (x5)      //load B0.scale for Matrix B0
  x5 = x5 + Bsstride
  mlde8 m5, x8, (x7)      //load B1.scale for Matrix B1
  x7 = x7 + Bsstride

  for(k0=0; k0<scale_vec_group; k0+=1)   //exhaust the scale element in the scale
matrix register
  {
    mlde8 m0, x10, (x9)   //load A0
    x9 = x9 + Astride
    mlde8 m2, x12, (x11)  //load B0
    x11 = x11 + Bstride
    mlde8 m4, x14, (x13)  //laod B1
    x13 = x13 + Bstride

    mfmacc.s.mxe4m3 m6, m2, m3, m0, m1    //C0 += A0*B0
    mfmacc.s.mxe4m3 m7, m4, m5, m0, m1    //C1 += A0*B1

    x15 = x15 + scale_vec_size
    mxcfg.colidx x15, mxa, mxb   //update scale element col idx

  }
}

x1 = x1 - Cstride
mste32 m6 x2, (x1) //store C0
x1 = x1 + Cstride
mste32 m7 x2, (x1) //store C1
```

Example code for Matrix A and Matrix B with different blocksizes:

```
//RLEN=512bit
//Source Operand: MXFP8
//Matrix A Blocksize: 32
```

```
//Matrix B Blocksize: 64
//A: M*K = 16*K
//B: N*K = 32*K
//C: M*N = 16*32

a_block_size      = 32
a_scale_vec_size  = 2
a_scale_vec_group = 32

b_block_size      = 64
b_scale_vec_size  = 1
b_scale_vec_group = 64

mcfgmi (RLEN/32)
mcfgni (RLEN/32)
mcfgki (RLEN/8)

//config the xmxadesc blksize field to 00, blocksize is 32
mxcfgi.blksize blksize32, mxa
//config the xmxbdesc blksize field to 01, blocksize is 64
mxcfgi.blksize blksize64, mxb


x1 = Cbaseaddr
mlde32 m6, x2, (x1) //load C0
x1 = x1 + Cstride
mlde32 m7, x2, (x1) //load C1


for(k2=0; k2<K; k2+=b_block_size*b_scale_vec_size*b_scale_vec_group)
{
  //initialize xmxadesc.colidx and xmxbdesc.colidx at the beginning of Matrix B
scale register
  x15=0
  x16=0
  mxcfg.colidx x15, mxa, mxb

  mlde8 m3, x6, (x5)      //load B0.scale for Matrix B0
  x5 = x5 + Bsstride
  mlde8 m5, x8, (x7)      //load B1.scale for Matrix B1
  x7 = x7 + Bsstride

  for(k1=0; k1<b_scale_vec_group/a_scale_vec_group; k1+=1)
  {
    //initialize xmxadesc.colidx at the beginning of Matrix A scale register
    x15 = 0
    mxcfg.colidx x15, mxa

    mlde8 m1, x4, (x3)         //load A0.scale for Matrix A0
```

```
    x3 = x3 + Asstride


    for(k1=0; k0<a_scale_vec_group; k0+=1)
    {
      mlde8 m0, x10, (x9)   //load A0
      x9 = x9 + Astride
      mlde8 m2, x12, (x11)  //load B0
      x11 = x11 + Bstride
      mlde8 m4, x14, (x13)  //laod B1
      x13 = x13 + Bstride

      mfmacc.s.mxfp8 m6, m2, m3, m0, m1    //C0 += A0*B0
      mfmacc.s.mxfp8 m7, m4, m5, m0, m1    //C1 += A0*B1


      x15 = x15 + a_scale_vec_size
      x16 = x16 + b_scale_vec_size
      mxcfg.colidx x15, mxa               //update mxa col idx
      mxcfg.colidx x16, mxb               //update mxb col idx
    }
  }
}

x1 = x1 - Cstride
mste32 m6 x2, (x1) //store C0
x1 = x1 + Cstride
mste32 m7 x2, (x1) //store C1
```

Chapter 7. Matrix Memory Model

Matrix memory instructions appear to execute in program order on the local hart.

Memory operations for each element are unordered within the instruction.

Matrix memory instructions follow RVWMO at the instruction level. Except for when a pair of read access the same locations, and at least one of the reads is generated by matrix instructions, they will not necessarily appear to execute in the same order by other processors.

Chapter 8. Instruction List

8.1. Configuration Instructions

Table 7. Configuration Instructions

Format	31	30 28	27 25	24 20	19 15	14 12	11 9	8	7	6 0
	type	**func**	**uop**	**rs2**	**rs1**	**func3**	**nop**	**mxb**	**mxa**	**opcode**
th.mcfgki	0	000	111	{uimm7[6:0],000}		000	000	0	0	major opcode
th.mcfgmi	0	001	111	{uimm7[6:0],000}		000	000	0	0	major opcode
th.mcfgni	0	010	111	{uimm7[6:0],000}		000	000	0	0	major opcode
th.mxcfgi.blksize	0	011	111	00000	{uimm2[1:0],000}	000	000	mxb	mxa	major opcode
th.mxcfgi.colidx	0	100	111	{uimm7[6:0],000}		000	000	mxb	mxa	major opcode
th.mcfgk	1	000	111	00000	rs1	000	000	0	0	major opcode
th.mcfgm	1	001	111	00000	rs1	000	000	0	0	major opcode
th.mcfgn	1	010	111	00000	rs1	000	000	0	0	major opcode
th.mcfg	1	111	111	00000	rs1	000	000	0	0	major opcode
th.mxcfg	1	011	111	00000	rs1	000	000	mxb	mxa	major opcode
th.mxcfg.blksize	1	100	111	00000	rs1	000	000	mxb	mxa	major opcode
th.mxcfg.colidx	1	101	111	00000	rs1	000	000	mxb	mxa	major opcode
th.mrelease	0	111	111	00000	00000	000	000	0	0	major opcode

8.2. Matrix Multiply instructions

Table 8. Matrix Multiply instructions

Format	31 28	27 25	24	23 21	20 18	17 15	14 12	11 10	9 7	6 0
	func4	**uop**	**size**	**ms2**	**ms1**	**md/ ms3**	**fun c3**	**size**	**type**	**opcode**
th.mfmacc.h	0001	000	0	ms2	ms1	md/ ms3	000	01	000	major opcode
th.mfmacc.s	0001	000	0	ms2	ms1	md/ ms3	000	10	000	major opcode
th.mfmacc.s.bf20	0001	000	0	ms2	ms1	md/ ms3	000	10	001	major opcode
th.mfmacc.d	0001	000	0	ms2	ms1	md/ ms3	000	11	000	major opcode
th.mfmacc.s.h	0001	000	1	ms2	ms1	md/ ms3	000	01	000	major opcode
th.mfmacc.s.bf16	0001	000	1	ms2	ms1	md/ ms3	000	01	100	major opcode
th.mfmacc.d.s	0001	000	1	ms2	ms1	md/ ms3	000	10	000	major opcode
th.mfmacc.h.e5m2	0001	000	1	ms2	ms1	md/ ms3	000	00	000	major opcode
th.mfmacc.h.e4m3	0001	000	1	ms2	ms1	md/ ms3	000	00	001	major opcode
th.mfmacc.bf16.e5m2	0001	000	1	ms2	ms1	md/ ms3	000	00	010	major opcode
th.mfmacc.bf16.e4m3	0001	000	1	ms2	ms1	md/ ms3	000	00	011	major opcode
th.mfmacc.s.e5m2	0001	000	1	ms2	ms1	md/ ms3	000	00	100	major opcode
th.mfmacc.s.e4m3	0001	000	1	ms2	ms1	md/ ms3	000	00	101	major opcode
th.mmacc.w.b	0010	000	0	ms2	ms1	md/ ms3	000	00	000	major opcode
th.mmaccu.w.b	0010	000	0	ms2	ms1	md/ ms3	000	00	001	major opcode
th.mmaccus.w.b	0010	000	0	ms2	ms1	md/ ms3	000	00	010	major opcode

th.mmaccsu.w.b	0010	000	0	ms2	ms1	md/ms3	000	00	011	major opcode
th.mmacc.w.p	0010	000	1	ms2	ms1	md/ms3	000	00	000	major opcode
th.mmaccu.w.p	0010	000	1	ms2	ms1	md/ms3	000	00	001	major opcode
th.mmaccus.w.p	0010	000	1	ms2	ms1	md/ms3	000	00	010	major opcode
th.mmaccsu.w.p	0010	000	1	ms2	ms1	md/ms3	000	00	011	major opcode
th.mmacc.w.bp	0010	000	0/1	ms2	ms1	md/ms3	000	10	000	major opcode
th.mmaccu.w.bp	0010	000	0/1	ms2	ms1	md/ms3	000	10	001	major opcode
th.mmaccus.w.bp	0010	000	0/1	ms2	ms1	md/ms3	000	10	010	major opcode
th.mmaccsu.w.bp	0010	000	0/1	ms2	ms1	md/ms3	000	10	011	major opcode
th.mfmacc.h.hb	0011	000	0	ms2	ms1	md/ms3	000	00	000	major opcode
th.mfmacc.h.hp	0011	000	0	ms2	ms1	md/ms3	000	01	000/001	major opcode
th.mfmaccu.h.hb	0011	000	1	ms2	ms1	md/ms3	000	00	000	major opcode
th.mfmaccu.h.hp	0011	000	1	ms2	ms1	md/ms3	000	01	000-001	major opcode
th.mfmacc.s.hb	0011	000	0	ms2	ms1	md/ms3	000	10	000/001	major opcode
th.mfmacc.s.hp	0011	000	0	ms2	ms1	md/ms3	000	11	000-011	major opcode
th.mfmaccu.s.hb	0011	000	1	ms2	ms1	md/ms3	000	10	000/001	major opcode
th.mfmaccu.s.hp	0011	000	1	ms2	ms1	md/ms3	000	11	000-011	major opcode
th.mfmacc.h.e4m3e2m1	0011	000	0	ms2	ms1	md/ms3	000	00	100	major opcode

th.mfmacc.h.e5m2e2m1	0011	000	0	ms2	ms1	md/ms3	000	01	100	major opcode
th.mfmacc.s.e4m3e2m1	0011	000	0	ms2	ms1	md/ms3	000	10	100/101	major opcode
th.mfmacc.s.e5m2e2m1	0011	000	0	ms2	ms1	md/ms3	000	11	100/101	major opcode
th.mfmacc.h.e2m1	0100	000	0	ms2	ms1	md/ms3	000	00	000	major opcode
th.mfmacc.bf16.e2m1	0100	000	0	ms2	ms1	md/ms3	000	00	001	major opcode
th.mfmacc.s.e2m1	0100	000	0	ms2	ms1	md/ms3	000	00	010	major opcode

8.3. Microscaling Matrix Multiply instructions

Table 9. Matrix Multiply instructions

Format	31	30 28	27 25	24	23 21	20 18	17 15	14 12	11 10	9 7	6 0
	nop	**ms2.s**	**ms1.s**	**nop**	**ms2**	**ms1**	**md/ms3**	**func3**	**imm**	**type**	**opcode**
th.mfmacc.s.mxe5m2	0	ms2.s	ms1.s	0	ms2	ms1	md/ms3	010	00	000	major opcode
th.mfmacc.s.mxe4m3	0	ms2.s	ms1.s	0	ms2	ms1	md/ms3	010	00	001	major opcode
th.mfmacc.s.mxe2m1	0	ms2.s	ms1.s	0	ms2	ms1	md/ms3	010	00	010	major opcode
th.mfmacc.s.mxe2m1.ue4m3	0	ms2.s	ms1.s	0	ms2	ms1	md/ms3	010	00	011	major opcode
th.mfmacc.s.mxe5m2mxe2m1	0	ms2.s	ms1.s	0	ms2	ms1	md/ms3	010	00/01	100	major opcode
th.mfmacc.s.mxe4m3mxe2m1	0	ms2.s	ms1.s	0	ms2	ms1	md/ms3	010	00/01	101	major opcode

8.4. Load/Store Instructions

Table 10. Load/Store Instructions

Format	31 28	27 25	24 20	19 15	14 12	11 10	9 7	6 0
	func4	**uop**	**rs2**	**rs1**	**func3**	**size**	**md/ms3**	**opcode**
th.mld<e8/e16/e32/e64>	0000	100	rs2	rs1	000	size	md	major opcode
th.mst<e8/e16/e32/e64>	0000	101	rs2	rs1	000	size	ms3	major opcode
th.msld<e8/e16/e32/e64>	0001	100	rs2	rs1	000	size	md	major opcode
th.msst<e8/e16/e32/e64>	0001	101	rs2	rs1	000	size	ms3	major opcode
th.mld<1/2/4/8>m<e8/e16/e32/e64>	0010	100	{00,nf}	rs1	000	size	md	major opcode
th.mst<1/2/4/8>m<e8/e16/e32/e64>	0010	101	{00,nf}	rs1	000	size	ms3	major opcode
th.mldt<e8/e16/e32/e64>	0011	100	rs2	rs1	000	size	md	major opcode
th.mstt<e8/e16/e32/e64>	0011	101	rs2	rs1	000	size	ms3	major opcode
th.msldt<e8/e16/e32/e64>	0100	100	rs2	rs1	000	size	md	major opcode
th.msstt<e8/e16/e32/e64>	0100	101	rs2	rs1	000	size	ms3	major opcode
th.mpld<e8/e16/e32/e64>	0101	100	rs2	rs1	000	size	000	major opcode
th.mpldt<e8/e16/e32/e64>	0110	100	rs2	rs1	000	size	000	major opcode

8.5. MISC Instructions

Table 11. MISC Instructions

Format	31 28	27 25	24	23 21	20 18	17 15	14 12	11 10	9 7	6 0
	func4	**uop**	**ctrl**	**ms2**	**ms1**	**md**	**func3**	**size**	**uimm3**	**opcode**
th.mmov.mm	0000	000	0	000	ms1	md	000	00	001	major opcode
th.mmov.mv.i	0000	010	0	000	ms1	md	000	00	uimm3	major opcode

th.mzero	1010	000	0	000	000	md	000	00	uimm3	major opcode
th.mmovb.x.m	0000	110	0	ms2	{0, rs1}		000	rd		major opcode
th.mmovh.x.m	0000	110	0	ms2	{1, rs1}		000	rd		major opcode
th.mmovw.x.m	0000	110	1	ms2	{0, rs1}		000	rd		major opcode
th.mmovd.x.m	0000	110	1	ms2	{1, rs1}		000	rd		major opcode
th.mdupb.m.x	0001	110	{rs2 , 00000 }				000	00	md	major opcode
th.mduph.m.x	0001	110	{rs2 , 00000 }				000	01	md	major opcode
th.mdupw.m.x	0001	110	{rs2 , 00000 }				000	10	md	major opcode
th.mdupd.m.x	0001	110	{rs2 , 00000 }				000	11	md	major opcode
th.mmovb.m.x	0010	110	{rs2 , rs1 }				000	00	md	major opcode
th.mmovh.m.x	0010	110	{rs2 , rs1 }				000	01	md	major opcode
th.mmovw.m.x	0010	110	{rs2 , rs1 }				000	10	md	major opcode
th.mmovd.m.x	0010	110	{rs2 , rs1 }				000	11	md	major opcode
th.mpack.mm	0011	110	0	ms2	ms1	md	000	00	000	major opcode
th.mpackhl.mm	0011	110	0	ms2	ms1	md	000	00	010	major opcode
th.mpackhh.mm	0011	110	0	ms2	ms1	md	000	00	011	major opcode
th.mrslidedown	0100	110	0	000	ms1	md	000	00	uimm3	major opcode
th.mrslideup	0100	110	0	001	ms1	md	000	00	uimm3	major opcode
th.mcslidedown.b	0100	110	0	010	ms1	md	000	00	uimm3	major opcode
th.mcslidedown.h	0100	110	0	010	ms1	md	000	01	uimm3	major opcode

th.mcslidedown.w	0100	110	0	010	ms1	md	000	10	uimm3	major opcode
th.mcslidedown.d	0100	110	0	010	ms1	md	000	11	uimm3	major opcode
th.mcslideup.b	0100	110	0	011	ms1	md	000	00	uimm3	major opcode
th.mcslideup.h	0100	110	0	011	ms1	md	000	01	uimm3	major opcode
th.mcslideup.w	0100	110	0	011	ms1	md	000	10	uimm3	major opcode
th.mcslideup.d	0100	110	0	011	ms1	md	000	11	uimm3	major opcode
th.mcmovb.mv.i	0101	110	0	000	ms1	md	000	00	uimm3	major opcode
th.mcmovh.mv.i	0101	110	0	000	ms1	md	000	01	uimm3	major opcode
th.mcmovw.mv.i	0101	110	0	000	ms1	md	000	10	uimm3	major opcode
th.mcmovd.mv.i	0101	110	0	000	ms1	md	000	11	uimm3	major opcode

8.6. Element-wise Instructions

Table 12. Element-wise Instructions

Format	31 28	27 25	24	23 21	20 18	17 15	14 12	11 10	9 7	6 0
	func4	**uop**	**ctrl**	**ms2**	**ms1**	**md**	**func3**	**size**	**uimm3**	**opcode**
th.msrl.w.mm	0000	000	0	ms2	ms1	md	001	10	000	major opcode
th.msrl.w.mv.i	0000	010	0	ms2	ms1	md	001	10	uimm3	major opcode
th.msll.w.mm	0000	000	1	ms2	ms1	md	001	10	000	major opcode
th.msll.w.mv.i	0000	010	1	ms2	ms1	md	001	10	uimm3	major opcode

th.mmax.w.mm	0001	000	0	ms2	ms1	md	001	10	000	major opcode
th.mmax.w.mv.i	0001	010	0	ms2	ms1	md	001	10	uimm3	major opcode
th.mumax.w.mm	0001	000	1	ms2	ms1	md	001	10	000	major opcode
th.mumax.w.mv.i	0001	010	1	ms2	ms1	md	001	10	uimm3	major opcode
th.mmin.w.mm	0010	000	0	ms2	ms1	md	001	10	000	major opcode
th.mmin.w.mv.i	0010	010	0	ms2	ms1	md	001	10	uimm3	major opcode
th.mumin.w.mm	0010	000	1	ms2	ms1	md	001	10	000	major opcode
th.mumin.w.mv.i	0010	010	1	ms2	ms1	md	001	10	uimm3	major opcode
th.madd.w.mm	0011	000	0	ms2	ms1	md	001	10	000	major opcode
th.madd.w.mv.i	0011	010	0	ms2	ms1	md	001	10	uimm3	major opcode
th.msub.w.mm	0100	000	0	ms2	ms1	md	001	10	000	major opcode
th.msub.w.mv.i	0100	010	0	ms2	ms1	md	001	10	uimm3	major opcode
th.msra.w.mm	0101	000	0	ms2	ms1	md	001	10	000	major opcode
th.msra.w.mv.i	0101	010	0	ms2	ms1	md	001	10	uimm3	major opcode
th.mn4clipl.w.mm	0110	000	0	ms2	ms1	md	001	10	000	major opcode
th.mn4cliph.w.mm	0110	000	1	ms2	ms1	md	001	10	000	major opcode
th.mn4clipl.w.mv.i	0110	010	0	ms2	ms1	md	001	10	uimm3	major opcode
th.mn4cliph.w.mv.i	0110	010	1	ms2	ms1	md	001	10	uimm3	major opcode

th.mn4cliplu.w.mm	0111	000	0	ms2	ms1	md	001	10	000	major opcode
th.mn4cliphu.w.mm	0111	000	1	ms2	ms1	md	001	10	000	major opcode
th.mn4cliplu.w.mv.i	0111	010	0	ms2	ms1	md	001	10	uimm3	major opcode
th.mn4cliphu.w.mv.i	0111	010	1	ms2	ms1	md	001	10	uimm3	major opcode
th.mmul.w.mm	1000	000	0	ms2	ms1	md	001	10	000	major opcode
th.mmul.w.mv.i	1000	010	0	ms2	ms1	md	001	10	uimm3	major opcode
th.mmulh.w.mm	1001	000	0	ms2	ms1	md	001	10	000	major opcode
th.mmulh.w.mv.i	1001	010	0	ms2	ms1	md	001	10	uimm3	major opcode
th.mscvtl.b.p	1010	000	1	000	ms1	md	001	00	000	major opcode
th.mscvth.b.p	1010	010	1	000	ms1	md	001	00	000	major opcode
th.mucvtl.b.p	1010	000	1	000	ms1	md	001	00	001	major opcode
th.mucvth.b.p	1010	010	1	000	ms1	md	001	00	001	major opcode
th.mfadd.h.mm	0000	001	0	ms2	ms1	md	001	01	0	major opcode
th.mfadd.h.mv.i	0000	011	0	ms2	ms1	md	001	01	uimm3	major opcode
th.mfsub.h.mm	0001	001	0	ms2	ms1	md	001	01	0	major opcode
th.mfsub.h.mv.i	0001	011	0	ms2	ms1	md	001	01	uimm3	major opcode
th.mfmul.h.mm	0010	001	0	ms2	ms1	md	001	01	0	major opcode
th.mfmul.h.mv.i	0010	011	0	ms2	ms1	md	001	01	uimm3	major opcode

th.mfmax.h.mm	0011	001	0	ms2	ms1	md	001	01	0	major opcode
th.mfmax.h.mv.i	0011	011	0	ms2	ms1	md	001	01	uimm3	major opcode
th.mfmin.h.mm	0100	001	0	ms2	ms1	md	001	01	0	major opcode
th.mfmin.h.mv.i	0100	011	0	ms2	ms1	md	001	01	uimm3	major opcode
th.mfadd.s.mm	0000	001	0	ms2	ms1	md	001	10	0	major opcode
th.mfadd.s.mv.i	0000	011	0	ms2	ms1	md	001	10	uimm3	major opcode
th.mfsub.s.mm	0001	001	0	ms2	ms1	md	001	10	0	major opcode
th.mfsub.s.mv.i	0001	011	0	ms2	ms1	md	001	10	uimm3	major opcode
th.mfmul.s.mm	0010	001	0	ms2	ms1	md	001	10	0	major opcode
th.mfmul.s.mv.i	0010	011	0	ms2	ms1	md	001	10	uimm3	major opcode
th.mfmax.s.mm	0011	001	0	ms2	ms1	md	001	10	0	major opcode
th.mfmax.s.mv.i	0011	011	0	ms2	ms1	md	001	10	uimm3	major opcode
th.mfmin.s.mm	0100	001	0	ms2	ms1	md	001	10	0	major opcode
th.mfmin.s.mv.i	0100	011	0	ms2	ms1	md	001	10	uimm3	major opcode
th.mfadd.d.mm	0000	001	0	ms2	ms1	md	001	11	0	major opcode
th.mfadd.d.mv.i	0000	011	0	ms2	ms1	md	001	11	uimm3	major opcode
th.mfsub.d.mm	0001	001	0	ms2	ms1	md	001	11	0	major opcode
th.mfsub.d.mv.i	0001	011	0	ms2	ms1	md	001	11	uimm3	major opcode

th.mfmul.d.mm	0010	001	0	ms2	ms1	md	001	11	0	major opcode
th.mfmul.d.mv.i	0010	011	0	ms2	ms1	md	001	11	uimm3	major opcode
th.mfmax.d.mm	0011	001	0	ms2	ms1	md	001	11	0	major opcode
th.mfmax.d.mv.i	0011	011	0	ms2	ms1	md	001	11	uimm3	major opcode
th.mfmin.d.mm	0100	001	0	ms2	ms1	md	001	11	0	major opcode
th.mfmin.d.mv.i	0100	011	0	ms2	ms1	md	001	11	uimm3	major opcode
th.mfcvtl.h.s	0101	001	0	000	ms1	md	001	10	000	major opcode
th.mfcvth.h.s	0101	011	0	000	ms1	md	001	10	000	major opcode
th.mfcvtl.s.d	0101	001	0	000	ms1	md	001	11	000	major opcode
th.mfcvth.s.d	0101	011	0	000	ms1	md	001	11	000	major opcode
th.mfcvtl.bf16.s	0101	001	0	000	ms1	md	001	10	100	major opcode
th.mfcvth.bf16.s	0101	011	0	000	ms1	md	001	10	100	major opcode
th.mfcvtl.e4m3.h	0101	001	0	000	ms1	md	001	01	010	major opcode
th.mfcvth.e4m3.h	0101	011	0	000	ms1	md	001	01	010	major opcode
th.mfcvtl.e5m2.h	0101	001	0	000	ms1	md	001	01	001	major opcode
th.mfcvth.e5m2.h	0101	011	0	000	ms1	md	001	01	001	major opcode
th.mfcvtl.s.h	0101	001	1	000	ms1	md	001	01	000	major opcode
th.mfcvth.s.h	0101	011	1	000	ms1	md	001	01	000	major opcode

th.mfcvtl.s.bf16	0101	001	1	000	ms1	md	001	01	100	major opcode
th.mfcvth.s.bf16	0101	011	1	000	ms1	md	001	01	100	major opcode
th.mfcvtl.d.s	0101	001	1	000	ms1	md	001	10	000	major opcode
th.mfcvth.d.s	0101	011	1	000	ms1	md	001	10	000	major opcode
th.mfcvtl.h.e4m3	0101	001	1	000	ms1	md	001	00	010	major opcode
th.mfcvth.h.e4m3	0101	011	1	000	ms1	md	001	00	010	major opcode
th.mfcvtl.h.e5m2	0101	001	1	000	ms1	md	001	00	001	major opcode
th.mfcvth.h.e5m2	0101	011	1	000	ms1	md	001	00	001	major opcode
th.mfcvtl.e2m1.e5m2	0101	101	0	000	ms1	md	001	00	001	major opcode
th.mfcvth.e2m1.e5m2	0101	111	0	000	ms1	md	001	00	001	major opcode
th.mfcvtl.e2m1.e4m3	0101	101	0	000	ms1	md	001	00	010	major opcode
th.mfcvth.e2m1.e4m3	0101	111	0	000	ms1	md	001	00	010	major opcode
th.mfcvtl.e5m2.e2m1	0101	101	1	000	ms1	md	001	00	001	major opcode
th.mfcvth.e5m2.e2m1	0101	111	1	000	ms1	md	001	00	001	major opcode
th.mfcvtl.e4m3.e2m1	0101	101	1	000	ms1	md	001	00	010	major opcode
th.mfcvth.e4m3.e2m1	0101	111	1	000	ms1	md	001	00	010	major opcode
th.mfcvtl.e4m3.s	0101	001	0	000	ms1	md	001	10	010	major opcode
th.mfcvth.e4m3.s	0101	011	0	000	ms1	md	001	10	010	major opcode

th.mfcvtl.e5m2.s	0101	001	0	000	ms1	md	001	10	001	major opcode
th.mfcvth.e5m2.s	0101	011	0	000	ms1	md	001	10	001	major opcode
th.mufcvt.s.w	0110	001	0	000	ms1	md	001	10	000	major opcode
th.mufcvtl.h.b	0110	001	0	001	ms1	md	001	00	000	major opcode
th.mufcvth.h.b	0110	011	0	001	ms1	md	001	00	000	major opcode
th.msfcvt.s.w	0110	001	0	100	ms1	md	001	10	000	major opcode
th.msfcvtl.h.b	0110	001	0	101	ms1	md	001	00	000	major opcode
th.msfcvth.h.b	0110	011	0	101	ms1	md	001	00	000	major opcode
th.mfucvt.w.s	0110	001	0	010	ms1	md	001	10	000	major opcode
th.mfucvtl.b.h	0110	001	0	011	ms1	md	001	01	000	major opcode
th.mfucvth.b.h	0110	011	0	011	ms1	md	001	01	000	major opcode
th.mfscvt.w.s	0110	001	0	110	ms1	md	001	10	000	major opcode
th.mfscvtl.b.h	0110	001	0	111	ms1	md	001	01	000	major opcode
th.mfscvth.b.h	0110	011	0	111	ms1	md	001	01	000	major opcode
th.msfcvtl.e4m3.p	0110	001	1	101	ms1	md	001	00	010	major opcode
th.msfcvtl.e5m2.p	0110	001	1	101	ms1	md	001	00	001	major opcode
th.msfcvth.e4m3.p	0110	011	1	101	ms1	md	001	00	010	major opcode
th.msfcvth.e5m2.p	0110	011	1	101	ms1	md	001	00	001	major opcode

th.mufcvtl.e4m3.p	0110	001	1	001	ms1	md	001	00	010	major opcode
th.mufcvtl.e5m2.p	0110	001	1	001	ms1	md	001	00	001	major opcode
th.mufcvth.e4m3.p	0110	011	1	001	ms1	md	001	00	010	major opcode
th.mufcvth.e5m2.p	0110	011	1	001	ms1	md	001	00	001	major opcode